

```
local_irq_disable();  
/* 禁止中断 */  
local_irq_enable();
```

这两个函数通常以单个汇编指令来实现（当然，这依赖于体系结构）。实际上，在 x86 中，`local_irq_disable()` 仅仅是 `cli` 指令，而 `local_irq_enable()` 只不过是 `sti` 指令。`cli` 和 `sti` 分别是对 `clear` 和 `set` 允许中断（`allow interrupt`）标志的汇编调用。换句话说，在发出中断的处理器上，它们将禁止和激活中断的传递。

如果在调用 `local_irq_disable()` 例程之前已经禁止了中断，那么该例程往往会带来潜在的危险；同样相应的 `local_irq_enable()` 例程也存在潜在危险，因为它将无条件地激活中断，尽管这些中断可能在开始时就是关闭的。所以我们需要一种机制把中断恢复到以前的状态而不是简单地禁止或激活。内核普遍关心这点是因为，内核中一个给定的代码路径既可以在中断激活的情况下达到，也可以在中断禁止的情况下达到，这取决于具体的调用链。例如，想象一下前面的代码片段是一个大函数的组成部分。这个函数被另外两个函数调用：其中一个函数禁止中断，而另一个函数不禁止中断。因为随着内核的不断增长，要想知道到达这个函数的所有代码路径将变得越来越困难，因此，在禁止中断之前保存中断系统的状态会更加安全一些。相反，在准备激活中断时，只需把中断恢复到它们原来的状态。

```
unsigned long flags;  
  
local_irq_save(flags); /* 禁止中断 */  
/* ... */  
local_irq_restore(flags); /* 中断被恢复到它们原来的状态 */
```

这些方法至少部分要以宏的形式实现，因此表面上 `flags` 参数（这些参数必须定义为 `unsigned long` 类型）是以值传递的。该参数包含具体体系结构的数据，也就是包含中断系统的状态。至少有一种体系结构把栈信息与值相结合（SPARC），因此 `flags` 不能传递给另一个函数（特别是它必须驻留在同一栈帧中）。基于这个原因，对 `local_irq_save()` 和对 `local_irq_restore()` 的调用必须在同一个函数中进行。

前面的所有函数既可以在中断中调用，也可以在进程上下文中调用。

不再使用全局的 `cli()`

以前的内核中提供了一种“能够禁止系统中所有处理器上的中断”方法。而且，如果另一个处理器调用这个方法，那么它就不得不等待，直到中断重新被激活才能继续执行。这个函数就是 `cli()`，相应的激活中断函数为 `sti()`——虽然适用于所有体系结构，但完全以 x86 为中心。这些接口在 2.5 版本开发期间被取消了，相应地，所有的中断同步现在必须结合使用本地中断控制和自旋锁（在第 9 章中进行讨论）。这就意味着，为了确保对共享数据的互斥访问，以前代码仅仅需要通过全局禁止中断达到互斥，而现在则需要多做些工作了。

以前，驱动程序编写者可能假定在他们的中断处理程序中，任何访问共享数据地方都可以使用 `cli()` 提供互斥访问。`cli()` 调用将确保没有其他的中断处理程序（因而只有它们特定的处理程序）会运行。此外，如果另一个处理器进入了 `cli` 保护区，那么它不可能继续运行，直到原来的处理器退出它们的 `cli()` 保护区，并调用了 `sti()` 后才能继续运行。

取消全局 cli() 有不少优点。首先, 强制驱动程序编写者实现真正的加锁。要知道具有特定目的细粒度锁比全局锁要快许多, 而且也完全吻合 cli() 的使用初衷。其次, 这也使得很多代码更具流线型, 避免了代码的成簇布局。所以由此得到的中断系统更简单也更易于理解。

7.9.2 禁止指定中断线

在前面的内容中, 我们看到了禁止整个处理器上所有中断的函数。在某些情况下, 只禁止整个系统中一条特定的中断线就够了。这就是所谓的屏蔽掉 (masking out) 一条中断线。作为例子, 你可能想在对中断的状态操作之前禁止设备中断的传递。为此, Linux 提供了四个接口:

```
void disable_irq(unsigned int irq);
void disable_irq_nosync(unsigned int irq);
void enable_irq(unsigned int irq);
void synchronize_irq(unsigned int irq);
```

前两个函数禁止中断控制器上指定的中断线, 即禁止给定中断向系统中所有处理器的传递。另外, 函数只有在当前正在执行的所有处理程序完成后, disable_irq() 才能返回。因此, 调用者不仅要确保不在指定线上传递新的中断, 同时还要确保所有已经开始执行的程序已全部退出。函数 disable_irq_nosync() 不会等待当前中断处理程序执行完毕。

函数 synchronize_irq() 等待一个特定的中断处理程序的退出。如果该处理程序正在执行, 那么该函数必须退出后才能返回。

对这些函数的调用可以嵌套。但要记住在一条指定的中断线上, 对 disable_irq() 或 disable_irq_nosync() 的每次调用, 都需要相应地调用一次 enable_irq()。只有在对 enable_irq() 完成最后一次调用后, 才真正重新激活了中断线。例如, 如果 disable_irq() 被调用了两次, 那么直到第二次调用 enable_irq() 后, 才能真正地激活中断线。

所有这三个函数可以从中断或进程上下文中调用, 而且不会睡眠。但如果从中断上下文中调用, 就要特别小心! 例如, 当你正在处理一条中断线时, 并不想激活它 (回想当某个处理程序的中断线正在被处理时, 它被屏蔽掉)。

禁止多个中断处理程序共享的中断线是不合适的。禁止中断线也就禁止了这条线上所有设备的中断传递。因此, 用于新设备的驱动程序应该倾向于不使用这些接口^①。根据规范, PCI 设备必须支持中断线共享, 因此, 它们根本不应该使用这些接口。所以, disable_irq() 及其相关函数在老式传统设备 (如 PC 并口) 的驱动程序中更容易被找到。

7.9.3 中断系统的状态

通常有必要了解中断系统的状态 (如中断是禁止的还是激活的), 或者你当前是否正处于中断上下文的执行状态中。

宏 irq_disable() 定义在 <asm/system.h> 中。如果本地处理器上的中断系统被禁止, 则它返

^① 很多老式设备, 尤其是 ISA 设备, 不提供方法检测它们是否产生了中断。因为这一点, ISA 的中断线常常不能共享。由于 PCI 规范要求中断共享, 因此, 现代基于 PCI 的设备支持中断共享。在当代计算机中, 几乎所有的中断线都可以共享。

回非 0；否则返回 0。

在 `<linux/hardirq.h>` 中定义的两个宏提供一个用来检查内核的当前上下文的接口，它们是：

```
in_interrupt()
in_irq()
```

第一个宏最有用：如果内核处于任何类型的中断处理中，它返回非 0，说明内核此刻正在执行中断处理程序，或者正在执行下半部处理程序。宏 `in_irq()` 只有在内核确实正在执行中断处理程序时才返回非 0。

通常情况下，你要检查自己是否处于进程上下文中。也就是说，你希望确保自己不在中断上下文中。这种情况很常见，因为代码要做一些像睡眠这样只能从进程上下文中做的事。如果 `in_interrupt()` 返回 0，则此刻内核处于进程上下文中。

是的，名字有点混淆，但可以对它们的含义稍加区别。表 7-2 是中断控制方法和其描述的摘要。

表 7-2 中断控制方法的列表

函 数	说 明
<code>local_irq_disable()</code>	禁止本地中断传递
<code>local_irq_enable()</code>	激活本地中断传递
<code>local_irq_save()</code>	保存本地中断传递的当前状态，然后禁止本地中断传递
<code>local_irq_restore()</code>	恢复本地中断传递到给定的状态
<code>disable_irq()</code>	禁止给定中断线，并确保该函数返回之前在该中断线上没有处理程序在运行
<code>disable_irq_nosync()</code>	禁止给定中断线
<code>enable_irq()</code>	激活给定中断线
<code>irqs_disabled()</code>	如果本地中断传递被禁止，则返回非 0；否则返回 0
<code>in_interrupt()</code>	如果在中断上下文中，则返回非 0；如果在进程上下文中，则返回 0
<code>in_irq()</code>	如果当前正在执行中断处理程序，则返回非 0；否则返回 0

7.10 小结

本章介绍了中断，它是一种由设备使用的硬件资源异步向处理器发信号。实际上，中断就是由硬件来打断操作系统。

大多数现代硬件都通过中断与操作系统通信。对给定硬件进行管理的驱动程序注册中断处理程序，是为了响应并处理来自相关硬件的中断。中断过程所做的工作包括应答并重新设置硬件，从设备拷贝数据到内存以及反之，处理硬件请求，并发送新的硬件请求。

内核提供的接口包括注册和注销中断处理程序、禁止中断、屏蔽中断线以及检查中断系统的状态。表 7-2 提供了这些函数的概述。

因为中断打断了其他代码的执行（进程，内核本身，甚至其他中断处理程序），它们必须赶快执行完。但通常是还有很多工作要做。为了在大量的工作与必须快速执行之间求得一种平衡，内核把处理中断的工作分为两半。中断处理程序，也就是上半部在本章讨论。现在，让我们了解下半部。

下半部和推后执行的工作

在第 7 章中，我们讨论了内核为处理中断而提供的中断处理程序机制。中断处理程序是内核中很有用的（实际上也是必不可少的）部分。但是，由于本身存在一些局限，所以它只能完成整个中断处理流程的上半部分。这些局限包括：

- 中断处理程序以异步方式执行，并且它有可能会打断其他重要代码（甚至包括其他中断处理程序）的执行。因此，为了避免被打断的代码停止时间过长，中断处理程序应该执行得越快越好。
- 如果当前有一个中断处理程序正在执行，在最好的情况下（如果 `IRQF_DISABLED` 设有被设置），与该中断同级的其他中断会被屏蔽，在最坏的情况下（如果设置了 `IRQF_DISABLED`），当前处理器上所有其他中断都会被屏蔽。因为禁止中断后硬件与操作系统无法通信，因此，中断处理程序执行得越快越好。
- 由于中断处理程序往往需要对硬件进行操作，所以它们通常有很高的时限要求。
- 中断处理程序不在进程上下文中运行，所以它们不能阻塞。这限制了它们所做的事情。

现在，为什么中断处理程序只能作为整个硬件中断处理流程一部分的原因就很明显了。操作系统必须有一个快速、异步、简单的机制负责对硬件做出迅速响应并完成那些时间要求很严格的操作。中断处理程序很适合于实现这些功能，可是，对于那些其他的、对时间要求相对宽松的任务，就应该推后到中断被激活以后再去运行。

这样，整个中断处理流程就被分为了两个部分，或叫两半。第一个部分是中断处理程序（上半部），就像我们在第 7 章讨论的那样，内核通过对它的异步执行完成对硬件中断的即时响应。在本章中，我们要研究的是中断处理流程中的另外那一部分，下半部（bottom halves）。

8.1 下半部

下半部的任务就是执行与中断处理密切相关但中断处理程序本身不执行的工作。在理想的情况下，最好是中断处理程序将所有工作都交给下半部分执行，因为我们希望在中断处理程序中完成的工作越少越好（也就是越快越好）。我们期望中断处理程序能够尽可能快地返回。

但是，中断处理程序注定要完成一部分工作。例如，中断处理程序几乎都需要通过操作硬件对中断的到达进行确认，有时它还会从硬件拷贝数据。因为这些工作对时间非常敏感，所以只能靠中断处理程序自己去完成。

剩下的几乎所有其他工作都是下半部执行的目标。例如，如果你在上半部中把数据从硬件拷贝到了内存，那么当然应该在下半部中处理它们。遗憾的是，并不存在严格明确的规定来说明到

底什么任务应该在哪个部分中完成——如何做决定完全取决于驱动程序开发者自己的判断。尽管在理论上不存在什么错误，但轻率的实现效果往往不很理想。记住，中断处理程序会异步执行，并且在最好的情况下它也会锁定当前的中断线。因此将中断处理程序持续执行的时间缩短到最小程度显得非常重要。对于在上半部和下半部之间划分工作，尽管不存在某种严格的规则，但还是有一些提示可供借鉴：

- 如果一个任务对时间非常敏感，将其放在中断处理程序中执行。
- 如果一个任务和硬件相关，将其放在中断处理程序中执行。
- 如果一个任务要保证不被其他中断（特别是相同的中断）打断，将其放在中断处理程序中执行。
- 其他所有任务，考虑放置在下半部执行。

当你开始尝试写自己的驱动程序的时候，读一下别人的中断处理程序和相应的下半部可能会让你受益匪浅。在决定怎样把你的中断处理流程中的工作划分到上半部和下半部中去的时候，问问自己什么必须放进上半部而什么可以放进下半部。通常，中断处理程序要执行得越快越好。

8.1.1 为什么要用下半部

理解为什么要让工作推后执行以及在什么时候推后执行非常关键。你希望尽量减少中断处理程序中需要完成的工作量，因为它在运行的时候，当前的中断线在所有处理器上都会被屏蔽。更糟糕的是，如果一个处理程序是 `IRQF_DISABLED` 类型，它执行的时候会禁止所有本地中断（而且把本地中断线全局地屏蔽掉）。而缩短中断被屏蔽的时间对系统的响应能力和性能都至关重要。再加上中断处理程序要与其他程序（甚至是其他的中断处理程序）异步执行，所以很明显，我们必须尽力缩短中断处理程序的执行。解决的方法就是把一些工作放到以后去做。

但具体放到以后什么时候去做呢？在这里，以后仅仅用来强调不是马上而已，理解这一点相当重要。下半部并不需要指明一个确切时间，只要把这些任务推迟一点，让它们在系统不太繁忙并且中断恢复后执行就可以了。通常下半部在中断处理程序一返回就会马上运行。下半部执行的关键在于当它们运行的时候，允许响应所有的中断。

不仅仅是 Linux，许多操作系统也把处理硬件中断的过程分为两个部分。上半部分简单快速，执行的时候禁止一些或者全部中断。下半部分（无论具体如何实现）稍后执行，而且执行期间可以响应所有的中断。这种设计可使系统处于中断屏蔽状态的时间尽可能的短，以此来提高系统的响应能力。

8.1.2 下半部的环境

和上半部只能通过中断处理程序实现不同，下半部可以通过多种机制实现。这些用来实现下半部的机制分别由不同的接口和子系统组成。在第7章中，我们了解到实现中断处理程序的方法只有一种^①，但在本章中你会发现，实现一个下半部会有许多不同的方法。实际上，在 Linux 发

^① 在 Linux 中，由于上半部从来都只能通过中断处理程序实现，所以它和中断处理程序可以说是等价的。——译者注

展的过程中曾经出现过多种下半部机制。让人备受困扰的是，其中不少机制名字起得很相像，甚至还有一些机制名字起得词不达意。这就需要专门的程序员来给下半部命名。

在本章中，我们将要讨论 2.6 版本的内核中的下半部机制是如何设计和实现的。同时我们也会讨论怎么在自己编写的内核代码中使用它们。而那些过去使用的、已经废除了有一段时间的机制，由于曾经闻名遐迩，所以在相关的时候我们还是会有所提及。

1. “下半部”的起源

最早的 Linux 只提供“bottom half”这种机制用于实现下半部。这个名字在那时毫无异义，因为当时它是将工作推后的唯一方法。这种机制也被称为“BH”，我们现在也这么叫它，以避免和“下半部”这个通用词汇混淆。像过往的那段美好岁月中的许多东西一样，BH 接口也非常简单。它提供了一个静态创建、由 32 个 bottom halves 组成的链表。上半部通过一个 32 位整数中的一位来标识出哪个 bottom half 可以执行。每个 BH 都在全局范围内进行同步。即使分属于不同的处理器，也不允许任何两个 bottom half 同时执行。这种机制使用方便却不够灵活，简单却有性能瓶颈。

2. 任务队列

不久，内核开发者们就引入了任务队列（task queue）机制来实现工作的推后执行，并用它来代替 BH 机制。内核为此定义了一组队列，其中每个队列都包含一个由等待调用的函数组成链表。根据其所处队列的位置，这些函数会在某个时刻执行。驱动程序可以把它们自己的下半部注册到合适的队列上去。这种机制表现得还不错，但仍不够灵活，没法代替整个 BH 接口。对于一些性能要求较高的子系统，像网络部分，它也不能胜任。

3. 软中断和 tasklet

在 2.3 这个开发版本中，内核开发者引入了软中断（softirqs）^①和 tasklet。如果无须考虑和过去开发的驱动程序兼容的话，软中断和 tasklet 可以完全代替 BH 接口^②。软中断是一组静态定义的下半部接口，有 32 个，可以在所有处理器上同时执行——即使两个类型相同也可以。tasklet 这一名称起得很糟糕，让人费解，它们是一种基于软中断实现的灵活性强、动态创建的下半部实现机制^③。两个不同类型的 tasklet 可以在不同的处理器上同时执行，但类型相同的 tasklet 不能同时执行。tasklet 其实是一种在性能和易用性之间寻求平衡的产物。对于大部分下半部处理来说，用 tasklet 就足够了，像网络这样对性能要求非常高的情况才需要使用软中断。可是，使用软中断需要特别小心，因为两个相同的软中断有可能同时被执行。此外，软中断还必须在编译期间就进行静态注册。与此相反，tasklet 可以通过代码进行动态注册。

有些人被这些概念彻底搞糊涂了，他们把所有的下半部都当成是软件产生的中断或软中断。换句话说，就是他们把软中断机制和下半部统统都叫软中断。别管他们好了。软中断与 BH 和 tasklet 并驾其名。

① 这里的软中断与第 4 章实现系统调用所提到的软中断（准确地说该叫它软件中断）指的是不同的概念。——译者注

② 把 BH 转化为软中断或者 tasklet 并不是轻而易举的事情，因为 BH 是全局同步的，因此，在执行期间假定没有其他 BH 执行。但是，这种转化最终还是在内核 2.5 中实现了。

③ 它们和进程没有一点关系。可以把一个 tasklet 当做一个简单易用的软中断。

在开发 2.5 版本的内核时，BH 接口最终被弃置了，所有的 BH 使用者必须转而使用其他下半部接口。此外，任务队列接口也被工作队列接口取代了。工作队列是一种简单但很有用的方法，它们先对要推后执行的工作排队，稍后在进程上下文中执行它们。稍后的内容中我们再来探究它们。

综上所述，在 2.6 这个当前版本中，内核提供了三种不同形式的下半部实现机制：软中断、tasklets 和工作队列。内核过去曾经用过的 BH 和任务队列接口，现在已经被湮没在记忆中了。

内核定时器

另外一个可以用于将工作推后执行的机制是内核定时器。不像本章到目前为止介绍到的所有这些机制，内核定时器把操作推迟到某个确定的时间段之后执行。也就是说，尽管本章讨论的其他机制可以把操作推后到除了现在以外的任何时间进行，但是当你必须保证在一个确定的时间段过去以后再运行时，你应该使用内核定时器。

较之本章讨论到的这些机制，定时器还有一些其他功能。有关定时器的详细内容在第 11 章中讨论。

4. 混乱的下半部概念

这些东西确实把人搅得很混乱，但它们其实只不过是一些起名的问题，让我们再来梳理一遍。

“下半部 (bottom half)” 是一个操作系统通用词汇，用于指代中断处理流程中推后执行的那一部分，之所以这样命名，是因为它表示中断处理方案一半的第二部分或者下半部。在 Linux 中，这个词目前确实就是这个含义。所有用于实现将工作推后执行的内核机制都被称为“下半部机制”。一些人错误地把所有的下半部机制都叫做“软中断”，真是在自寻烦恼。

“下半部”这个词也指代 Linux 最早提供的那种将工作推后执行的实现机制。由于该机制也被叫做“BH”，所以，我们就使用它的这个名称，而让“下半部”这个词仍然保持它通常的含义。BH 机制很早以前就被反对使用了，在 2.5 版内核中，它就被完全去除了。

当前，有三种机制可以用来实现将工作推后执行：软中断、tasklet 和工作队列。tasklet 通过软中断实现，而工作队列与它们完全不同。表 8-1 揭示了下半部机制的演化历程。

表 8-1 下半部状态

下半部机制	状 态
BH	在 2.5 中去除
任务队列 (Task queues)	在 2.5 中去除
软中断 (Softirq)	从 2.3 开始引入
tasklet	从 2.3 开始引入
工作队列 (Work queues)	从 2.5 开始引入

在搞清楚这些混乱的命名之后，让我们开始具体研究各个机制。

8.2 软中断

我们的讨论从实际的下半部实现——软中断方法开始。软中断使用得比较少；而 tasklet 是

下半部更常用的一种形式。但是，由于 tasklet 是通过软中断实现的，所以我们先来研究软中断。软中断的代码位于 kernel/softirq.c 文件中。

8.2.1 软中断的实现

软中断是在编译期间静态分配的。它不像 tasklet 那样能被动态地注册或注销。软中断由 softirq_action 结构表示，它定义在 <linux/interrupt.h> 中：

```
struct softirq_action {  
    void (*action)(struct softirq_action *);  
};
```

kernel/softirq.c 中定义了一个包含有 32 个该结构体的数组。

```
static struct softirq_action softirq_vec[NR_SOFTIRQS];
```

每个被注册的软中断都占据该数组的一项，因此最多可能有 32 个软中断。注意，这是一个定值——注册的软中断数目的最大值没法动态改变。在当前版本的内核中，这 32 个项中只用到 9 个^①。

1. 软中断处理程序

软中断处理程序 action 的函数原型如下：

```
void softirq_handler(struct softirq_action *)
```

当内核运行一个软中断处理程序的时候，它就会执行这个 action 函数，其唯一的参数为指向相应 softirq_action 结构体的指针。例如，如果 my_softirq 指向 softirq_vec 数组的某项，那么内核会用如下的方式调用软中断处理程序中的函数：

```
my_softirq->action(my_softirq);
```

当你看到内核把整个结构体都传递给软中断处理程序而不是仅仅传递数据值的时候，你可能会很吃惊。这个小技巧可以保证将来在结构体中加入新的域时，无须对所有的软中断处理程序都进行变动。如果需要，软中断处理程序可以方便地解析它的参数，从数据成员中提取数值。

一个软中断不会抢占另外一个软中断。实际上，唯一可以抢占软中断的是中断处理程序。不过，其他的软中断（甚至是相同类型的软中断）可以在其他处理器上同时执行。

2. 执行软中断

一个注册的软中断必须在被标记后才会执行。这被称作触发软中断（raising the softirq）。通常，中断处理程序会在返回前标记它的软中断，使其在稍后被执行。于是，在合适的时刻，该软中断就会运行。在下列地方，待处理的软中断会被检查和执行：

- 从一个硬件中断代码处返回时
- 在 ksoftirqd 内核线程中
- 在那些显式检查和执行待处理的软中断的代码中，如网络子系统中

^① 大部分驱动程序都使用 tasklet 来实现它们的下半部。我们将在下面的内容中看到，tasklet 是用软中断实现的。

不管是用什么办法唤起，软中断都要在 `do_softirq()` 中执行。该函数很简单。如果有待处理的软中断，`do_softirq()` 会循环遍历每一个，调用它们的处理程序。让我们观察一下 `do_softirq()` 经过简化后的核心部分：

```
u32 pending;

pending = local_softirq_pending();
if (pending) {
    struct softirq_action *h;

    /* 重设待处理的位图 */
    set_softirq_pending(0);

    h = softirq_vec;
    do {
        if (pending & 1)
            h->action(h);
        h++;
        pending >>= 1;
    } while (pending);
}
```

以上摘录的是软中断处理的核心部分。它检查并执行所有待处理的软中断，具体要做的包括：

- 1) 用局部变量 `pending` 保存 `local_softirq_pending()` 宏的返回值。它是待处理的软中断的 32 位位图——如果第 n 位被设置为 1，那么第 n 位对应类型的软中断等待处理。
- 2) 现在待处理的软中断位图已经被保存，可以将实际的软中断位图清零了^①。
- 3) 将指针 `h` 指向 `softirq_vec` 的第一项。
- 4) 如果 `pending` 的第一位被置为 1，则 `h->action(h)` 被调用。
- 5) 指针加 1，所以现在它指向 `softirq_vec` 数组的第二项。
- 6) 位掩码 `pending` 右移一位。这样会丢弃第一位，然后让其他各位依次向右移动一个位置。于是，原来的第二位现在就在第一位的位置上了（依次类推）。
- 7) 现在指针 `h` 指向数组的第二项，`pending` 位掩码的第二位现在也到了第一位上。重复执行上面的步骤。
- 8) 一直重复下去，直到 `pending` 变为 0，这表明已经没有待处理的软中断了，我们的任务也就完成了。注意，这种检查足以保证 `h` 总指向 `softirq_vec` 的有效项，因为 `pending` 最多只可能设置 32 位，循环最多也只能执行 32 次。

① 实际上在执行此步操作时需要禁止本地中断。但在这个简化的例子中被省略了。如果中断不被屏蔽，在保存位图和清除它的间隙，可能会有一个新的软中断被唤醒（它自然也就等待处理）。这可能会造成对此待处理的位进行不应该的清 0。

8.2.2 使用软中断

软中断保留给系统中对时间要求最严格以及最重要的下半部使用。目前，只有两个子系统（网络和 SCSI）直接使用软中断。此外，内核定时器和 tasklet 都是建立在软中断上的。如果你想加入一个新的软中断，首先应该问问自己为什么用 tasklet 实现不了。tasklet 可以动态生成，由于它们对加锁的要求不高，所以使用起来也很方便，而且它们的性能也非常不错。当然，对于时间要求严格并能自己高效地完成加锁工作的应用，软中断会是正确的选择。

1. 分配索引

在编译期间，通过在 `<linux/interrupt.h>` 中定义的一个枚举类型来静态地声明软中断。内核用这些从 0 开始的索引来表示一种相对优先级。索引号小的软中断在索引号大的软中断之前执行。

建立一个新的软中断必须在此枚举类型中加入新的项。而加入时，你不能像在其他地方一样，简单地把新项加到列表的末尾。相反，你必须根据希望赋予它的优先级来决定加入的位置。习惯上，`HI_SOFTIRQ` 通常作为第一项，而 `RCU_SOFTIRQ` 作为最后一项。新项可能插在 `BLOCK_SOFTIRQ` 和 `TASKLET_SOFTIRQ` 之间。表 8-2 列举出了已有的 tasklet 类型。

表 8-2 tasklet 类型列表

tasklet	优先级	软中断描述
<code>HI_SOFTIRQ</code>	0	优先级高的 tasklets
<code>TIMER_SOFTIRQ</code>	1	定时器的下半部
<code>NET_TX_SOFTIRQ</code>	2	发送网络数据包
<code>NET_RX_SOFTIRQ</code>	3	接收网络数据包
<code>BLOCK_SOFTIRQ</code>	4	BLOCK 装置
<code>TASKLET_SOFTIRQ</code>	5	正常优先权的 tasklets
<code>SCHED_SOFTIRQ</code>	6	调度程度
<code>HRTIMER_SOFTIRQ</code>	7	高分辨率定时器
<code>RCU_SOFTIRQ</code>	8	RCU 锁定

2. 注册你的处理程序

接着，在运行时通过调用 `open_softirq()` 注册软中断处理程序，该函数有两个参数：软中断的索引号和处理函数。如网络子系统，在 `net/coredev.c` 通过以下方式注册自己的软中断：

```
open_softirq(NET_TX_SOFTIRQ, net_tx_action);
open_softirq(NET_RX_SOFTIRQ, net_rx_action);
```

软中断处理程序执行的时候，允许响应中断，但它自己不能休眠。在一个处理程序运行的时候，当前处理器上的软中断被禁止。但其他的处理器仍可以执行别的软中断。实际上，如果同一个软中断在它被执行的同时再次被触发了，那么另外一个处理器可以同时运行其处理程序。这意味着任何共享数据（甚至是仅在软中断处理程序内部使用的全局变量）都需要严格的锁保护（在后面的内容中会讨论）。这点很重要，它也是 tasklet 更受青睐的原因。单纯地禁止你的软中断处理程序同时执行不是很理想。如果仅仅通过互斥的加锁方式来防止它自身的并发执行，那么使用软中断就没有任何意义了。因此，大部分软中断处理程序，都通过采取单处理器数据（仅属于某

一个处理器的数据，因此根本不需要加锁）或其他一些技巧来避免显式地加锁，从而提供更出色的性能。

引入软中断的主要原因是其可扩展性。如果不需要扩展到多个处理器，那么，就使用 tasklet 吧。tasklet 本质上也是软中断，只不过同一个处理程序的多个实例不能在多个处理器上同时运行。

3. 触发你的软中断

通过在枚举类型的列表中添加新项以及调用 `open_softirq()` 进行注册以后，新的软中断处理程序就能够运行。`raise_softirq()` 函数可以将一个软中断设置为挂起状态，让它在下次调用 `do_softirq()` 函数时投入运行。举个例子，网络子系统可能会调用：

```
raise_softirq(NET_TX_SOFTIRQ);
```

这会触发 `NET_TX_SOFTIRQ` 软中断。它的处理程序 `net_tx_action()` 就会在内核下一次执行软中断时投入运行。该函数在触发一个软中断之前先要禁止中断，触发后再恢复原来的状态。如果中断本来就已经被禁止了，那么可以调用另一函数 `raise_softirq_irqoff()`，这会带来一些优化效果。如：

```
/*
 * 中断已经被禁止
 */
raise_softirq_irqoff(NET_TX_SOFTIRQ);
```

在中断处理程序中触发软中断是最常见的形式。在这种情况下，中断处理程序执行硬件设备的相关操作，然后触发相应的软中断，最后退出。内核在执行完中断处理程序以后，马上就会调用 `do_softirq()` 函数。于是软中断开始执行中断处理程序留给它去完成的剩余任务。在这个例子中，“上半部”和“下半部”名字的含义一目了然。

8.3 tasklet

tasklet 是利用软中断实现的一种下半部机制。我们之前提到过，它和进程没有任何关系。tasklet 和软中断在本质上很相似，行为表现也相近，但是，它的接口更简单，锁保护也要求较低。

选择到底是用软中断还是 tasklet 其实很简单：通常你应该用 tasklet。就像我们在前面看到的，软中断的使用者屈指可数。它只在那些执行频率很高和连续性要求很高的情况下才需要使用。而 tasklet 却有更广泛的用途。大多数情况下用 tasklet 效果都不错，而且它们还非常容易使用。

8.3.1 tasklet 的实现

因为 tasklet 是通过软中断实现的，所以它们本身也是软中断。前面讨论过了，tasklet 由两类软中断代表：`HI_SOFTIRQ` 和 `TASKLET_SOFTIRQ`。这两者之间唯一的实际区别在于，`HI_SOFTIRQ` 类型的软中断先于 `TASKLET_SOFTIRQ` 类型的软中断执行。

1. tasklet 结构体

tasklet 由 tasklet_struct 结构表示。每个结构体单独代表一个 tasklet，它在 <linux/interrupt.h> 中定义为：

```
struct tasklet_struct {
    struct tasklet_struct *next;    /* 链表中的下一个 tasklet */
    unsigned long state;           /* tasklet 的状态 */
    atomic_t count;                /* 引用计数器 */
    void (*func)(unsigned long);    /* tasklet 处理函数 */
    unsigned long data;            /* 给 tasklet 处理函数的参数 */
};
```

结构体中的 func 成员是 tasklet 的处理程序（像软中断中的 action 一样），data 是它唯一的参数。

state 成员只能在 0、TASKLET_STATE_SCHED 和 TASKLET_STATE_RUN 之间取值。TASKLET_STATE_SCHED 表明 tasklet 已被调度，正准备投入运行，TASKLET_STATE_RUN 表明该 tasklet 正在运行。TASKLET_STATE_RUN 只有在多处理器的系统上才会作为一种优化来使用，单处理器系统任何时候都清楚单个 tasklet 是不是正在运行（它要么就是当前正在执行的代码，要么不是）。

count 成员是 tasklet 的引用计数器。如果它不为 0，则 tasklet 被禁止，不允许执行；只有当它为 0 时，tasklet 才被激活，并且在被设置为挂起状态时，该 tasklet 才能够执行。

2. 调度 tasklet

已调度的 tasklet（等同于被触发的软中断）^①存放在两个单处理器数据结构：tasklet_vec（普通 tasklet）和 tasklet_hi_vec（高优先级的 tasklet）。这两个数据结构都是由 tasklet_struct 结构体构成的链表。链表中的每个 tasklet_struct 代表一个不同的 tasklet。

tasklet_struct 结构体构成的链表。链表中的每个 tasklet_struct 代表一个不同的 tasklet。

tasklet 由 tasklet_schedule() 和 tasklet_hi_schedule() 函数进行调度，它们接受一个指向 tasklet_struct 结构的指针作为参数。两个函数非常类似（区别在于一个使用 TASKLET_SOFTIRQ 而另一个用 HI_SOFTIRQ）。在接下来的内容中我们将仔细研究怎么编写和使用 tasklets。现在，让我们先考察一下 tasklet_schedule() 的细节：

tasklet_schedule() 的执行步骤：

1) 检查 tasklet 的状态是否为 TASKLET_STATE_SCHED。如果是，说明 tasklet 已经被调度过了^②，函数立即返回。

2) 调用 _tasklet_schedule()。

3) 保存中断状态，然后禁止本地中断。在我们执行 tasklet 代码时，这么做能够保证当 tasklet_schedule() 处理这些 tasklet 时，处理器上的数据不会弄乱。

4) 把需要调度的 tasklet 加到每个处理器一个的 tasklet_vec 链表或 tasklet_hi_vec 链表的表头上去。

① 此处是命名混乱的又一个实例。为什么软中断是唤醒而 tasklet 是调度？谁能说得清楚？两个词实际上都表示将此下半部设置为待执行状态后以便稍后执行。

② 有可能是一个 tasklet 已经被调度过但还没来得及执行，而该 tasklet 又被唤起了一次。——译者注

5) 唤起 TASKLET_SOFTIRQ 或 HI_SOFTIRQ 软中断, 这样在下一次调用 do_softirq() 时会执行该 tasklet。

6) 恢复中断到原状态并返回。

在前面的内容中我们曾经提到过挂起, do_softirq() 会尽可能早地在下一个合适的时机执行。由于大部分 tasklet 和软中断都是在中断处理程序中被设置成待处理状态, 所以最近一个中断返回的时候看起来就是执行 do_softirq() 的最佳时机。因为 TASKLET_SOFTIRQ 和 HI_SOFTIRQ 已经被触发了, 所以 do_softirq() 会执行相应的软中断处理程序。而这两个处理程序, tasklet_action() 和 tasklet_hi_action(), 就是 tasklet 处理的核心。让我们观察它们做了什么:

1) 禁止中断 (没有必要首先保存其状态, 因为这里的代码总是作为软中断被调用, 而且中断总是被激活的), 并为当前处理器检索 tasklet_vec 或 tasklet_hi_vec 链表。

2) 将当前处理器上的该链表设置为 NULL, 达到清空的效果。

3) 允许响应中断。没有必要再恢复它们回原状态, 因为这段程序本身就是作为软中断处理程序被调用的, 所以中断是应该被允许的。

4) 循环遍历获得链表上的每一个待处理的 tasklet。

5) 如果是多处理器系统, 通过检查 TASKLET_STATE_RUN 来判断这个 tasklet 是否正在其他处理器上运行。如果它正在运行, 那么现在就不要执行, 跳到下一个待处理的 tasklet 去 (回忆一下, 同一时间里, 相同类型的 tasklet 只能有一个执行)。

6) 如果当前这个 tasklet 没有执行, 将其状态设置为 TASKLET_STATE_RUN, 这样别的处理器就不会再去执行它了。

7) 检查 count 值是否为 0, 确保 tasklet 没有被禁止。如果 tasklet 被禁止了, 则跳到下一个挂起的 tasklet 去。

8) 我们已经清楚地知道这个 tasklet 没有在其他地方执行, 并且被我们设置成执行状态, 这样它在其他部分就不会被执行, 并且引用计数为 0, 现在可以执行 tasklet 的处理程序了。

9) tasklet 运行完毕, 清除 tasklet 的 state 域的 TASKLET_STATE_RUN 状态标志。

10) 重复执行下一个 tasklet, 直至没有剩余的等待处理的 tasklet。

tasklet 的实现很简单, 但非常巧妙。我们可以看到, 所有的 tasklet 都通过重复运用 HI_SOFTIRQ 和 TASKLET_SOFTIRQ 这两个软中断实现。当一个 tasklet 被调度时, 内核就会唤起这两个软中断中的一个。随后, 该软中断会被特定的函数处理, 执行所有已调度的 tasklet。这个函数保证同一时间里只有一个给定类别的 tasklet 会被执行 (但其他不同类型的 tasklet 可以同时执行)。所有这些复杂性都被一个简洁的接口隐藏起来了。

8.3.2 使用 tasklet

大多数情况下, 为了控制一个寻常的硬件设备, tasklet 机制都是实现自己的下半部的最佳选择。tasklet 可以动态创建, 使用方便, 执行起来也还算快。此外, 尽管它们的名字使人混淆, 但能加深你的印象: 那是逗人喜爱的。

1. 声明你自己的 tasklet

你既可以静态地创建 tasklet, 也可以动态地创建它。选择哪种方式取决于你到底是有 (或者

是想要) 一个对 tasklet 的直接引用还是间接引用。如果你准备静态地创建一个 tasklet (也就是有一个它的直接引用), 使用下面 <linux/interrupt.h> 中定义的两个宏中的一个:

```
DECLARE_TASKLET(name, func, data)
DECLARE_TASKLET_DISABLED(name, func, data);
```

这两个宏都能根据给定的名称静态地创建一个 tasklet_struct 结构。当该 tasklet 被调度以后, 给定的函数 func 会被执行, 它的参数由 data 给出。这两个宏之间的区别在于引用计数器的初始值设置不同。前面一个宏把创建的 tasklet 的引用计数器设置为 0, 该 tasklet 处于激活状态。另一个把引用计数器设置为 1, 所以该 tasklet 处于禁止状态。下面是一个例子:

```
DECLARE_TASKLET(my_tasklet, my_tasklet_handler, dev);
```

这行代码其实等价于

```
struct tasklet_struct my_tasklet = { NULL, 0, ATOMIC_INIT(0),
                                     my_tasklet_handler, dev};
```

这样就创建了一个名为 my_tasklet, 处理程序为 tasklet_handler 并且是已被激活的 tasklet。当处理程序被调用的时候, dev 就会被传递给它。

还可以通过将一个间接引用(一个指针)赋给一个动态创建的 tasklet_struct 结构的方式来初始化一个 tasklet_init():

```
tasklet_init(t, tasklet_handler, dev); /* 动态而不是静态创建 */
```

2. 编写你自己的 tasklet 处理程序

tasklet 处理程序必须符合规定的函数类型:

```
void tasklet_handler(unsigned long data)
```

因为是靠软中断实现, 所以 tasklet 不能睡眠。这意味着你不能在 tasklet 中使用信号量或者其他什么阻塞式的函数。由于 tasklet 运行时允许响应中断, 所以你必须做好预防工作(如屏蔽中断然后获取一个锁), 如果你的 tasklet 和中断处理程序之间共享了某些数据的话。两个相同的 tasklet 决不会同时执行, 这点和软中断不同——尽管两个不同的 tasklet 可以在两个处理器上同时执行。如果你的 tasklet 和其他的 tasklet 或者是软中断共享了数据, 你必须进行适当地锁保护。(参看第 9 章和第 10 章)。

3. 调度你自己的 tasklet

通过调用 tasklet_schedule() 函数并传递给它相应的 tasklet_struct 的指针, 该 tasklet 就会被调度以便执行:

```
tasklet_schedule(&my_tasklet); /* 把 my_tasklet 标记为挂起 */
```

在 tasklet 被调度以后, 只要有机会它就会尽可能早地运行。在它还没有得到运行机会之前, 如果有一个相同的 tasklet 又被调度了^①, 那么它仍然只会运行一次。而如果这时它已经开始运行

^① 这里应该是唤起的意思, 在前面讲述调度流程的内容里可以看到, 调度 tasklet 的第一个步骤就是检查是否重复, 所以这里根本不会完成调度。——译者注

了，比如说在另外一个处理器上，那么这个新的 tasklet 会被重新调度并再次运行。作为一种优化措施，一个 tasklet 总在调度它的处理器上执行——这是希望能更好地利用处理器的高速缓存。

你可以调用 `tasklet_disable()` 函数来禁止某个指定的 tasklet。如果该 tasklet 当前正在执行，这个函数会等到它执行完毕再返回。你也可以调用 `tasklet_disable_nosync()` 函数，它也用来禁止指定的 tasklet，不过它无须在返回前等待 tasklet 执行完毕。这么做往往不太安全，因为你无法估计该 tasklet 是否仍在执行。调用 `tasklet_enable()` 函数可以激活一个 tasklet，如果希望激活 `DECLARE_TASKLET_DISABLED()` 创建的 tasklet，你也得调用这个函数，如：

```
tasklet_disable(&my_tasklet);    /* tasklet 现在被禁止 */

/* 我们现在毫无疑问地知道 tasklet 不能运行 */

tasklet_enable(&my_tasklet);     /* tasklet 现在被激活 */
```

你可以通过调用 `tasklet_kill()` 函数从挂起的队列中去掉一个 tasklet。该函数的参数是一个指向某个 tasklet 的 `tasklet_struct` 的长指针。在处理一个经常重新调度它自身的 tasklet 的时候，从挂起的队列中移去已调度的 tasklet 会很有用。这个函数首先等待该 tasklet 执行完毕，然后再将它移去。当然，没有什么可以阻止其他地方的代码重新调度该 tasklet。由于该函数可能会引起休眠，所以禁止在中断上下文中使用它。

4. ksoftirqd

每个处理器都有一组辅助处理软中断（和 tasklet）的内核线程。当内核中出现大量软中断的时候，这些内核进程就会辅助处理它们。因为 tasklet 通过用软件中断实施，下面的讨论同样适用于软中断和 tasklet。简洁起见，我们将主要参考软中断。

我们前面曾经阐述过，对于软中断，内核会选择在几个特殊时机进行处理。而在中断处理程序返回时处理是最常见的。软中断被触发的频率有时可能很高（像在进行大流量的网络通信期间）。更不利的是，处理函数有时还会自行重复触发。也就是说，当一个软中断执行的时候，它可以重新触发自己以便再次得到执行（事实上，网络子系统就会这么做）。如果软中断本身出现的频率就高，再加上它们又有将自己重新设置为可执行状态的能力，那么就会导致用户空间进程无法获得足够的处理器时间，因而处于饥饿状态。而且，单纯的对重新触发的软中断采取不立即处理的策略，也无法让人接受。当软中断最初提出时，就是一个让人进退维谷的问题，亟待解决，而直观的解决方案又都不理想。首先，就让我们看看两种最容易想到的直观的方案。

第一种方案是，只要还有被触发并等待处理的软中断，本次执行就要负责处理，重新触发的软中断也在本次执行返回前被处理。这样做可以保证对内核的软中断采取即时处理的方式，关键在于，对重新触发的软中断也会立即处理。当负载很高的时候这样做就会出问题，此时会有大量被触发的软中断，而它们本身又会重复触发。系统可能会一直处理软中断，根本不能完成其他任务。用户空间的任務被忽略了——实际上，只有软中断和中断处理程序轮流执行，而系统的用户只能等待。只有在系统永远处于低负载的情况下，这种方案才会有理想的运行效果；只要系统有哪怕是中等程度的负载量，这种方案就无法让人满意。用户空间根本不能容忍有明显的停顿出现。

第二种方案选择不处理重新触发的软中断。在从中断返回的时候，内核和平常一样，也会检查所有挂起的软中断并处理它们。但是，任何自行重新触发的软中断都不会马上处理，它们被放

到下一个软中断执行时去处理。而这个时机通常也就是下一次中断返回的时候，这等于是说，一定得等一段时间，新的（或者重新触发的）软中断才能被执行。可是，在比较空闲的系统中，立即处理软中断才是比较好的做法。很不幸，这个方案显然又是一个时好时坏的选择。尽管它能保证用户空间不处于饥饿状态，但它却让软中断忍受饥饿的痛苦，而根本没有好好利用闲置的系统资源。

在设计软中断时，开发者就意识到需要一些折中。最终在内核中实现的方案是不会立即处理重新触发的软中断。而作为改进，当大量软中断出现的时候，内核会唤醒一组内核线程来处理这些负载。这些线程在最低的优先级上运行（nice 值是 19），这能避免它们跟其他重要的任务抢夺资源。但它们最终肯定会被执行，所以，这个折中方案能够保证在软中断负担很重的时候，用户程序不会因为得不到处理时间而处于饥饿状态。相应的，也能保证“过量”的软中断终究会得到处理。最后，在空闲系统上，这个方案同样表现良好，软中断处理得非常迅速（因为仅存的内核线程肯定会马上调度）。

每个处理器都有一个这样的线程。所有线程的名字都叫做 ksoftirqd/n，区别在于 n，它对应的是处理器的编号。在一个双 CPU 的机器上就有两个这样的线程，分别叫 ksoftirqd/0 和 ksoftirqd/1。为了保证只要有空闲的处理器，它们就会处理软中断，所以给每个处理器都分配一个这样的线程。一旦该线程被初始化，它就会执行类似下面这样的死循环：

```
for (;;) {
    if (!softirq_pending(cpu))
        schedule();

    set_current_state(TASK_RUNNING);

    while (softirq_pending(cpu)) {
        do_softirq();
        if (need_resched())
            schedule();
    }

    set_current_state(TASK_INTERRUPTIBLE);
}
```

只要有待处理的软中断（由 softirq_pending() 函数负责发现），ksoftirq 就会调用 do_softirq() 去处理它们。通过重复执行这样的操作，重新触发的软中断也会被执行。如果有必要的话，每次迭代后都会调用 schedule() 以便让更重要的进程得到处理机会。当所有需要执行的操作都完成以后，该内核线程将自己设置为 TASK_INTERRUPTIBLE 状态，唤起调度程序选择其他可执行进程投入运行。

只要 do_softirq() 函数发现已经执行过的内核线程重新触发了它自己，软中断内核线程就会被唤醒。

8.3.3 老的 BH 机制

尽管 BH 机制令人欣慰地退出了历史舞台，在 2.6 版内核中已经难觅踪迹了。可是，它毕竟

曾历经了漫长的时光——从最早版本的内核就开始了。由于其余威尚存，所以仅仅不经意地提起它是不够的，尽管在 2.6 版本中已经不再使用它了，但历史就是历史，应该被了解。

BH 很古老，但它能揭示一些东西。所有 BH 都是静态定义的，最多可以有 32 个。由于处理函数必须在编译时就被定义好，所以实现模块时不能直接使用 BH 接口。不过业已存在的 BH 倒是可以利用。随着时间的推移，这种静态要求和最大为 32 个的数目限制最终妨碍了它们的应用。

每个 BH 处理程序都严格地按顺序执行——不允许任何两个 BH 处理程序同时执行，即使它们的类型不同。这样做倒是使同步变得简单了，可是却不利于多处理器的可扩展性，也不利于大型 SMP 的性能。使用 BH 的驱动程序很难从多个处理器上受益，特别是网络层，可以说为此饱受困扰。

除了这些特点，BH 机制和 tasklet 就很像了。实际上，在 2.4 内核中，BH 就是基于 tasklet 实现的。所有可能的 32 个 BH 都通过在 `<linux/interrupt.h>` 中定义的常量表示。如果需要将一个 BH 标志为挂起状态，可以把相应的 BH 号传给 `mark_bh()` 函数。在 2.4 内核中，这将导致随后调度 BH tasklet，具体工作是由函数 `bh_action()` 完成的。而在 2.4 内核以前，BH 机制独立实现，不依赖任何低级 BH 机制，这和现在的软中断很像。

由于这种形式的下半部机制存在缺点，内核开发者们希望引入任务队列机制来代替它。尽管任务队列得到了不少使用者的认可，但它实际上并没有达成这个目的。在 2.3 版的内核中，引入新的软中断和 tasklet 机制也就结束了对 BH 的使用。BH 机制基于 tasklet 重新实现。不幸的是，因为新接口本身降低了对执行的序列化（serialization）保障，所以从 BH 接口移植到 tasklet 或软中断接口上操作起来非常复杂^①。在 2.5 版中，这种移植最终在定时器和 SCSI（最后的 BH 使用者）转换到软中断机制后完成了。于是内核开发者们立即除去了 BH 接口。终于解脱了，BH！

8.4 工作队列

工作队列（work queue）是另外一种将工作推后执行的形式，它和我们前面讨论的所有其他形式都不相同。工作队列可以把工作推后，交由一个内核线程去执行——这个下半部分总是会在进程上下文中执行。这样，通过工作队列执行的代码能占尽进程上下文的所有优势。最重要的就是工作队列允许重新调度甚至是睡眠。

通常，在工作队列和软中断 /tasklet 中做出选择非常容易。如果推后执行的任务需要睡眠，那么就选择工作队列。如果推后执行的任务不需要睡眠，那么就选择软中断或 tasklet。实际上，工作队列通常可以用内核线程替换。但是由于内核开发者们非常反对创建新的内核线程（在有些场合，使用这种冒失的方法可能会吃到苦头），所以我们也推荐使用工作队列。当然，这种接口也的确很容易使用。

如果你需要用一个可以重新调度的实体来执行你的下半部处理，你应该使用工作队列。它是

① 实际上，接口降低对执行的序列化保障能够提高安全性，但却难于编程。移植一个 BH 到 tasklet，需要仔细地斟酌代码与其他 tasklet 同时执行是否安全。不过，当最终完成这样的移植后，性能上的提高会使这些额外工作物有所值。

唯一能在进程上下文中运行的下半部实现机制，也只有它才可以睡眠。这意味着在你需要获得大量的内存时，在你需要获取信号量时，在你需要执行阻塞式的 I/O 操作时，它都会非常有用。如果你不需要用一个内核线程来推后执行工作，那么就考虑使用 tasklet 吧。

8.4.1 工作队列的实现

工作队列子系统是一个用于创建内核线程的接口，通过它创建的进程负责执行由内核其他部分排到队列里的任务。它创建的这些内核线程称作工作者线程（worker thread）。工作队列可以让你的驱动程序创建一个专门的工作者线程来处理需要推后的工作。不过，工作队列子系统提供了一个缺省的工作者线程来处理这些工作。因此，工作队列最基本的表现形式，就转变成了一个把需要推后执行的任务交给特定的通用线程的这样一种接口。

缺省的工作者线程叫做 events/n，这里 n 是处理器的编号；每个处理器对应一个线程。例如，单处理器的系统只有 events/0 这样一个线程，而双处理器的系统就会多一个 events/1 线程。缺省的工作者线程会从多个地方得到被推后的工作。许多内核驱动程序都把它们的下半部交给缺省的工作者线程去做。除非一个驱动程序或者子系统必须建立一个属于它自己的内核线程，否则最好使用缺省线程。

不过并不存在什么东西能够阻止代码创建属于自己的工作者线程。如果你需要在工作者线程中执行大量的处理操作，这样做或许会带来好处。处理器密集型和性能要求严格的任务会因为拥有自己的工作线程而获得好处。此时这么做也有助于减轻缺省线程的负担，避免工作队列中其他需要完成的工作处于饥饿状态。

1. 表示线程的数据结构

工作者线程用 workqueue_struct 结构表示：

```
/*
 * 外部可见的工作队列抽象是
 * 由每个 CPU 的工作队列组成的数组
 */
struct workqueue_struct {
    struct cpu_workqueue_struct cpu_wq[NR_CPUS];
    struct list_head list;
    const char *name;
    int singlethread;
    int freezeable;
    int rt;
};
```

该结构内是一个由 cpu_workqueue_struct 结构组成的数组，它定义在 kernel/workqueue.c 中，数组的每一项对应系统中的一个处理器。由于系统中每个处理器对应一个工作者线程，所以对于给定的某台计算机来说，就是每个处理器，每个工作者线程对应一个这样的 cpu_workqueue_struct 结构体。cpu_workqueue_struct 是 kernel/workqueue.c 中的核心数据结构：

```
struct cpu_workqueue_struct {
    spinlock_t lock;           /* 锁保护这种结构 */

    struct list_head worklist; /* 工作列表 */
};
```



```

wait_queue_head_t more_work;
struct work_struct*current_struct;

struct workqueue_struct *wq;      /* 关联工作队列结构 */
task_t *thread;                  /* 关联线程 */
};

```

注意，每个工作者线程类型关联一个自己的 `workqueue_struct`。在该结构体里面，给每个线程分配一个 `cpu_workqueue_struct`，因而也就是给每个处理器分配一个，因为每个处理器都有一个该类型的工作者线程。

2. 表示工作的数据结构

所有的工作者线程都是用普通的内核线程实现的，它们都要执行 `worker_thread()` 函数。在它初始化完以后，这个函数执行一个死循环并开始休眠。当有操作被插入到队列里的时候，线程就会被唤醒，以便执行这些操作。当没有剩余的操作时，它又会继续休眠。

工作用 `<linux/workqueue.h>` 中定义的 `work_struct` 结构体表示：

```

struct work_struct {
    atomic_long_t data;
    struct list_head entry;
    work_func_t func;
};

```

这些结构体被连接成链表，在每个处理器上的每种类型的队列都对应这样一个链表。比如，每个处理器上用于执行被推后的工作的那个通用线程就有一个这样的链表。当一个工作者线程被唤醒时，它会执行它的链表上的所有工作。工作被执行完毕，它就将相应的 `work_struct` 对象从链表上移去。当链表上不再有对象的时候，它就会继续休眠。

我们可以看一下 `worker_thread()` 函数的核心流程，简化如下：

```

for (;;) {
    prepare_to_wait(&cwq->more_work, &wait, TASK_INTERRUPTIBLE);
    if (list_empty(&cwq->worklist))
        schedule();
    finish_wait(&cwq->more_work, &wait);
    run_workqueue(cwq);
}

```

该函数在死循环中完成了以下功能：

- 1) 线程将自己设置为休眠状态（state 被设成 `TASK_INTERRUPTIBLE`），并把自己加入到等待队列中。
 - 2) 如果工作链表是空的，线程调用 `schedule()` 函数进入睡眠状态。
 - 3) 如果链表中有对象，线程不会睡眠。相反，它将自己设置成 `TASK_RUNNING`，脱离等待队列。
 - 4) 如果链表非空，调用 `run_workqueue()` 函数执行被推后的工作。
- 下一步，由 `run_workqueue()` 函数来实际完成推后到此的工作：

```

while (!list_empty(&cwq->worklist)) {
    struct work_struct *work;
    work_func_t f;
    void *data;

    work = list_entry(cwq->worklist.next, struct work_struct, entry);
    f = work->func;
    list_del_init(cwq->worklist.next);
    work_clear_pending(work);
    f(work);
}

```

该函数循环遍历链表上每个待处理的工作，执行链表每个节点上的 `workqueue_struct`[⊖] 中的 `func` 成员函数：

- 1) 当链表不为空时，选取下一个节点对象。
- 2) 获取我们希望执行的函数 `func` 及其参数 `data`。
- 3) 把该节点从链表上解下来，将待处理标志位 `pending` 清零。
- 4) 调用函数。
- 5) 重复执行。

3. 工作队列实现机制的总结

这些数据结构之间的关系确实让人觉得混乱，难以理清头绪。图 8-1 给出了示意图，把所有这些关系放在一起进行解释。

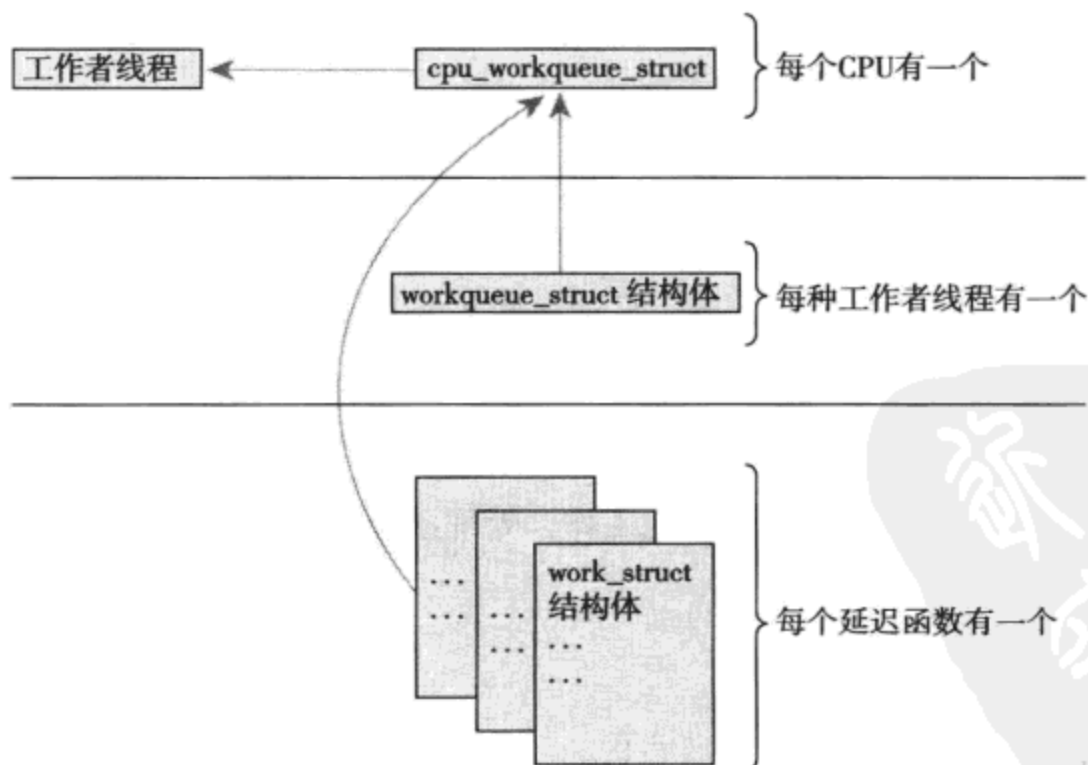


图 8-1 工作 (work)、工作队列和工作者线程之间的关系

位于最高一层的是工作者线程。系统允许有多种类型的工作者线程存在。对于指定的一个类

⊖ 应该是 `work_struct`。——译者注

型，系统的每个 CPU 上都有一个该类的工作者线程。内核中有些部分可以根据需要来创建工作者线程，而在默认情况下内核只有 event 这一类型的工作者线程。每个工作者线程都由一个 `cpu_workqueue_struct` 结构体表示。而 `workqueue_struct` 结构体则表示给定类型的所有工作者线程。

例如，除系统默认的通用 events 工作者类型之外，我自己还加入了一种 falcon 工作者类型。并且使用的是一个拥有四个处理器的计算机。那么，系统中现在有四个 event 类型的线程（因而也就有四个 `cpu_workqueue_struct` 结构体）和四个 falcon 类型的线程（因而会有另外四个 `cpu_workqueue_struct` 结构体）。同时，有一个对应 event 类型的 `workqueue_struct` 和一个对应 falcon 类型的 `workqueue_struct`。

工作处于最底层，让我们从这里开始。你的驱动程序创建这些需要推后执行的工作^①。它们用 `work_struct` 结构来表示。这个结构体中最重要的部分是一个指针，它指向一个函数，而正是该函数负责处理需要推后执行的具体任务。工作会被提交给某个具体的工作者线程——在这种情况下，就是特殊的 falcon 线程。然后这个工作者线程会被唤醒并执行这些排好的工作。

大部分驱动程序都使用的是现存的默认工作者线程。它们使用起来简单、方便。可是，在有些要求更严格的情况下，驱动程序需要自己的工作线程。比如说 XFS 文件系统就为自己创建了两新的工作者线程。

8.4.2 使用工作队列

工作队列的使用非常简单。我们先来看一下缺省的 events 任务队列，然后再看看创建新的工作者线程。

1. 创建推后的工作

首先要做的是实际创建一些需要推后完成的工作。可以通过 `DECLARE_WORK` 在编译时静态地建该结构体：

```
DECLARE_WORK(name, void (*func) (void *), void *data);
```

这样就会静态地创建一个名为 `name`，处理函数为 `func`，参数为 `data` 的 `work_struct` 结构体。同样，也可以在运行时通过指针创建一个工作：

```
INIT_WORK(struct work_struct *work, void(*func) (void *), void *data);
```

这会动态地初始化一个由 `work` 指向的工作，处理函数为 `func`，参数为 `data`。

2. 工作队列处理函数

工作队列处理函数的原型是：

```
void work_handler(void *data)
```

这个函数会由一个工作者线程执行，因此，函数会运行在进程上下文中。默认情况下，允许响应中断，并且不持有任何锁。如果需要，函数可以睡眠。需要注意的是，尽管操作处理函数运行在进程上下文中，但它不能访问用户空间，因为内核线程在用户空间没有相关的内存映射。通

^① 这其实可以理解成用“工作”这种接口封装我们实际需要推后的工作，以便后续的工作者线程处理。——译者注

常在发生系统调用时，内核会代表用户空间的进程运行，此时它才能访问用户空间，也只有在此时它才会映射用户空间的内存。

在工作队列和内核其他部分之间使用锁机制就像在其他的进程上下文中使用锁机制一样方便。这使编写处理函数变得相对容易。第 9 章和第 10 章会讨论到锁机制。

3. 对工作进行调度

现在工作已经被创建，我们可以调度它了。想要把给定工作的处理函数提交给缺省的 events 工作线程，只需调用：

```
schedule_work(&work);
```

`work` 马上就会被调度，一旦其所在的处理器上的工作者线程被唤醒，它就会被执行。

有时候你并不希望工作马上就被执行，而是希望它经过一段延迟以后再执行。在这种情况下，你可以调度它在指定的时间执行：

```
schedule_delayed_work(&work, delay);
```

这时，`&work` 指向的 `work_struct` 直到 `delay` 指定的时钟节拍用完以后才会执行。在第 10 章将介绍这种使用时钟节拍作为时间单位的方法。

4. 刷新操作

排入队列的工作会在工作者线程下一次被唤醒的时候执行。有时，在继续下一步工作之前，你必须保证一些操作已经执行完毕了。这一点对模块来说就很重要，在卸载之前，它就有可能需要调用下面的函数。而在内核的其他部分，为了防止竞争条件的出现，也可能需要确保不再有待处理的工作。

出于以上目的，内核准备了一个用于刷新指定工作队列的函数：

```
void flush_scheduled_work(void);
```

函数会一直等待，直到队列中所有对象都被执行以后才返回。在等待所有待处理的工作执行的时候，该函数会进入休眠状态，所以只能在进程上下文中使用它。

注意，该函数并不取消任何延迟执行的工作。就是说，任何通过 `schedule_delayed_work()` 调度的工作，如果其延迟时间未结束，它并不会因为调用 `flush_scheduled_work()` 而被刷新掉。取消延迟执行的工作应该调用：

```
int cancel_delayed_work(struct work_struct *work);
```

这个函数可以取消任何与 `work_struct` 相关的挂起工作。

5. 创建新的工作队列

如果缺省的队列不能满足你的需要，你应该创建一个新的工作队列和与之相应的工作者线程。由于这么做会在每个处理器上都创建一个工作者线程，所以只有在你明确了必须要靠自己的一套线程来提高性能的情况下，再创建自己的工作队列。

创建一个新的任务队列和与之相关的工作者线程，你只需调用一个简单的函数：

```
struct workqueue_struct *create_workqueue(const char *name);
```

`name` 参数用于该内核线程的命名。比如，缺省的 `events` 队列的创建就调用的是：

```
struct workqueue_struct *keventd_wq ;
keventd_wq = create_workqueue("events");
```

这样就会创建所有的工作者线程（系统中的每个处理器都有一个），并且做好所有开始处理工作之前的准备工作。

创建一个工作的时候无须考虑工作队列的类型。在创建之后，可以调用下面列举的函数。这些函数与 `schedule_work()` 以及 `schedule_delayed_work()` 相近，唯一的区别就在于它们针对给定的工作队列而不是缺省的 `events` 队列进行操作。

```
int queue_work(struct workqueue_struct *wq, struct work_struct *work)
```

```
int queue_delayed_work(struct workqueue_struct *wq,
                      struct work_struct *work,
                      unsigned long delay)
```

最后，你可以调用下面的函数刷新指定的工作队列：

```
flush_workqueue(struct workqueue_struct *wq);
```

该函数和前面讨论过的 `flush_scheduled_work()` 作用相同，只是它在返回前等待清空的是给定的队列。

8.4.3 老的任务队列机制

像 BH 接口被软中断和 tasklet 代替一样，由于任务队列接口存在的种种缺陷，它也被工作队列接口取代了。像 tasklet 一样，任务队列接口（内核中常常称作 tq）其实也和进程没有什么相关之处^①。任务队列接口的使用者在 2.5 开发版中分为两部分，其中一部分转向了使用 tasklet，还有另外一部分继续使用任务队列接口。而目前任务队列接口剩余的部分已经演化成了工作队列接口。由于任务队列在内核中曾经使用过一段时间，出于了解历史的目的，我们对它进行一个大体回顾。

任务队列机制通过定义一组队列来实现其功能。每个队列都有自己的名字，比如调度程序队列、立即队列和定时器队列。不同的队列在内核中的不同场合使用。keventd 内核线程负责执行调度程序队列的相关任务。它是整个工作队列接口的先驱。定时器队列会在系统定时器的每个时间节拍时执行，而立即队列能够得到双倍的运行机会，以保证它能“立即”执行。当然，还有其他一些队列。此外，你还可以动态地创建自己的新队列。

这些听起来都挺有用，但任务队列接口实际上是一团乱麻。这些队列基本上都是些随意创建的抽象概念，散落在内核各处，就像飘散在空气中。唯有调度队列有点意义，它能用来把工作推后到进程上下文完成。

任务队列的另一好处就是接口特别简单。如果不考虑这些队列的数量和执行时随心所欲的规

① 下半部的各种命名简直可以算得上是迷惑内核开发新手的撒手锏。老实说，这些名字简直就是噩梦！

则，它的接口确实够简单。但这也就是全部意义所在了——任务队列剩下的东西乏善可陈。

许多任务队列接口的使用者都已经转向使用其他的下半部实现机制了，大部分选择了 tasklet，只有调度程序队列的使用者在苦苦支撑。最终，keventd 代码演化成了我们今天使用的工作队列机制，而任务队列最终退出了历史舞台。

8.5 下半部机制的选择

在各种不同的下半部实现机制之间做出选择是很重要的。在当前的 2.6 版内核中，有三种可能的选择：软中断、tasklet 和工作队列。tasklet 基于软中断实现，所以两者很相近。工作队列机制与它们完全不同，它靠内核线程实现。

从设计的角度考虑，软中断提供的执行序列化的保障最少。这就要求软中断处理函数必须格外小心地采取一些步骤确保共享数据的安全，两个甚至更多相同类别的软中断有可能在不同的处理器上同时执行。如果被考察的代码本身多线索化的工作就做得非常好，比如网络子系统，它完全使用单处理器变量，那么软中断就是非常好的选择。对于时间要求严格和执行频率很高的应用来说，它执行得也最快。

如果代码多线索化考虑得并不充分，那么选择 tasklet 意义更大。它的接口非常简单，而且，由于两个同种类型的 tasklet 不能同时执行，所以实现起来也会简单一些。tasklet 是有效的软中断，但不能并发运行。驱动程序开发者应当尽可能选择 tasklet 而不是软中断，当然，如果准备利用每一处理器上的变量或者类似的情形，以确保软中断能安全地在多个处理器上并发地运行，那么还是选择软中断。

如果你需要把任务推后到进程上下文中完成，那么在这三者中就只能选择工作队列了。如果进程上下文并不是必须的条件（明确点说，就是如果并不需要睡眠），那么软中断和 tasklet 可能更合适。工作队列造成的开销最大，因为它要牵扯到内核线程甚至是上下文切换。这并不是说工作队列的效率低，如果每秒钟有几千次中断，就像网络子系统时常经历的那样，那么采用其他的机制可能更合适一些。尽管如此，针对大部分情况，工作队列都能提供足够的支持。

如果讲到易于使用，工作队列就当仁不让了。使用缺省的 events 队列简直不费吹灰之力。接下来是 tasklet，它的接口也很简单。最后才是软中断，它必须静态创建，并且需要慎重考虑其实现。

表 8-3 是对三种下半部接口的比较。

表 8-3 对下半部的比较

下半部	上下文	顺序执行保障
软中断	中断	没有
tasklet	中断	同类型不能同时执行
工作队列	进程	没有（和进程上下文一样被调度）

简单地说，一般的驱动程序的编写者需要做两个选择。首先，你是不是需要一个可调度的实体来执行需要推后完成的工作——从根本上来说，你有休眠的需要吗？要是有的话，工作队列就是你的唯一选择。否则最好用 tasklet。要是必须专注于性能的提高，那么就考虑软中断吧。

8.6 在下半部之间加锁

到现在为止，我们还没讨论过锁机制，这是一个非常有趣且广泛的话题，我将在第9章和第10章里仔细讨论它。不过，在这里还是应该对它的重要性有所了解：在使用下半部机制时，即使是在一个单处理器的系统上，避免共享数据被同时访问也是至关重要的。记住，一个下半部实际上可能在任何时候执行。如果你对锁机制一无所知的话，你也可以在读完第9章和第10章以后再回过头来看这部分。

使用 tasklet 的一个好处在于，它自己负责执行的序列化保障：两个相同类型的 tasklet 不允许同时执行，即使在不同的处理器上也不行。这意味着你无须为 intra-tasklet[⊖] 的同步问题操心了。tasklet 之间的同步（就是当两个不同类型的 tasklet 共享同一数据时）需要正确使用锁机制。

如果进程上下文和一个下半部共享数据，在访问这些数据之前，你需要禁止下半部的处理并得到锁的使用权。做这些是为了本地和 SMP 的保护并且防止死锁的出现。

如果中断上下文和一个下半部共享数据，在访问数据之前，你需要禁止中断并得到锁的使用权。所做的这些也是为了本地和 SMP 的保护并且防止死锁的出现。

任何在工作队列中被共享的数据也需要使用锁机制。其中有关锁的要点和在一般内核代码中没什么区别，因为工作队列本来就是在进程上下文中执行的。

在第9章里，我们会揭示锁的奥妙。而在第10章中，我们将讲述内核的加锁原语。这两章会描述如何保护下半部使用的数据。

8.7 禁止下半部

一般单纯禁止下半部的处理是不够的。为了保证共享数据的安全，更常见的做法是，先得到一个锁然后再禁止下半部的处理。驱动程序中通常使用的都是这种方法，在第10章会详细介绍。然而，如果你编写的是内核的核心代码，你也可能仅需要禁止下半部就可以了。

如果需要禁止所有的下半部处理（明确点说，就是所有的软中断和所有的 tasklet），可以调用 local_bh_disable() 函数。允许下半部进行处理，可以调用 local_bh_enable() 函数。没错，这些函数的命名也有问题；可是既然 BH 接口早就让位给软中断了，那么谁又会去改这些名称呢？表 8-4 是这些函数的一份摘要。

表 8-4 下半部机制控制函数的清单

函 数	描 述
void local_bh_disable()	禁止本地处理器的软中断和 tasklet 的处理
void local_bh_enable()	激活本地处理器的软中断和 tasklet 的处理

这些函数有可能被嵌套使用——最后被调用的 local_bh_enable() 最终激活下半部。比如，第一次调用 local_bh_disable()，则本地软中断处理被禁止；如果 local_bh_disable() 被调用三次，则

⊖ 这个词语是我造的。——译者注

本地处理仍然被禁止；只有当第四次调用 `local_bh_enable()` 时，软中断处理才被重新激活。

函数通过 `preempt_count`（很有意思，还是这个计数器，内核抢占的时候用的也是它）为每个进程维护一个计数器[⊖]。当计数器变为 0 时，下半部才能够被处理。因为下半部的处理已经被禁止，所以 `local_bh_enable()` 还需要检查所有现存的待处理的下半部并执行它们。

这些函数与硬件体系结构相关，它们位于 `<asm/softirq.h>` 中，通常由一些复杂的宏实现。下面是为那些好奇的人准备了 C 语言的近似描述：

```
/*
 * 通过增加 preempt_count 禁止本地下半部
 */
void local_bh_disable(void)
{
    struct thread_info *t = current_thread_info();
    t->preempt_count += SOFTIRQ_OFFSET;
}
/*
 * 减少 preempt_count 如果该返回值为 0，将导致自动激活下半部
 *
 * 执行挂起下半部
 */
void local_bh_enable(void)
{
    struct thread_info *t = current_thread_info();
    t->preempt_count -= SOFTIRQ_OFFSET;
    /*
     * preempt_count 是否为 0，另外是否有挂起下半部，如果都满足，则执行待执
     * 行的下半部
     */
    if (unlikely(!t->preempt_count && softirq_pending(smp_processor_id())))
        do_softirq();
}
```

这些函数并不能禁止工作队列的执行。因为工作队列是在进程上下文中运行的，不会涉及异步执行的问题，所以也就没有必要禁止它们执行。由于软中断和 tasklet 是异步发生的（就是说，在中断处理返回的时候），所以，内核代码必须禁止它们。另一方面，对于工作队列来说，它保护共享数据所做的工作和其他任何进程上下文中所做的都差不多。第 9 章和第 10 章将揭示其中的细节。

8.8 小结

在本章中，我们涵盖了用于延迟 Linux 内核工作的三种机制：软中断、tasklet 和工作队列。我们考察了其设计和实现，讨论了如何把这些机制应用到代码中，也调侃了易于混淆的命名。为了完整起见，我们也考察了曾经的下半部机制：BH 和任务队列——这些用在以前的 Linux 内核版本中。

[⊖] 实际上，中断和下半部子系统都用到了这个计数器。其实，在 Linux 中，每个任务都有的单个计数器代表了任务的原子性。在类似调试 `sleeping-while-atomic` 之类的错误时，这种做法已被证明是非常有效的。

因为下半部中相当程度地用到了同步和并发，所以本章谈了很多相关的话题。我们甚至围绕本章还讨论了禁止下半部的问题，这是由并发保护引起的，这一话题到此只是刚刚引入。第9章将从理论上讨论内核同步和并发，为理解这一问题的本质打下基础。第10章将讨论我们心爱的内核为解决这一问题所提供的具体接口。以这两章为基石，你的梦想就得以实现。

数字资源
PDG

第 ⑨ 章

内核同步介绍

在使用共享内存的应用程序中，程序员必须特别留意保护共享资源，防止共享资源并发访问。内核也不例外。共享资源之所以要防止并发访问，是因为如果多个执行线程^①同时访问和操作数据，就有可能发生各线程之间相互覆盖共享数据的情况，造成被访问数据处于不一致状态。并发访问共享数据是造成系统不稳定的一类隐患，而且这种错误一般难以跟踪和调试——所以首先应该认识到这个问题的重要性。

要做到对共享资源的恰当保护往往很困难。多年之前，在 Linux 还未支持对称多处理器的时候，避免并发访问数据的方法相对来说比较简单。在单一处理器的时候，只有在中断发生的时候，或在内核代码明确地请求重新调度、执行另一个任务的时候，数据才可能被并发访问。因此早期内核开发工作相比如今要简单许多！

但当年的太平日子一去不复返了，从 2.0 版开始，内核就开始支持对称多处理器了，而且从那以后对它的支持不断地加强和完善。支持多处理器意味着内核代码可以同时运行在两个或更多的处理器上。因此，如果不加保护，运行在两个不同处理器上的内核代码完全可能在同一时刻里并发访问共享数据。随着 2.6 版内核的出现，Linux 内核已发展成抢占式内核，这意味着（当然，还是指不加保护的情况下）调度程序可以在任何时刻抢占正在运行的内核代码，重新调度其他的进程执行。现在，内核代码中有不少部分都能够同步执行，而它们都必须妥善地保护起来。

本章我们先提纲挈领地讨论操作系统内核中的并发和同步问题，第 10 章我们将详细介绍 Linux 内核为解决同步问题和防止产生竞争条件而提供的机制及接口。

9.1 临界区和竞争条件

所谓临界区（也称为临界段）就是访问和操作共享数据的代码段。多个执行线程并发访问同一个资源通常是不安全的，为了避免在临界区中并发访问，编程者（也就是你）必须保证这些代码原子地执行——也就是说，操作在执行结束前不可被打断，就如同整个临界区是一个不可分割的指令一样。如果两个执行线程有可能处于同一个临界区中同时执行，那么这就是程序包含的一个 bug。如果这种情况确实发生了，我们就称它是竞争条件（race conditions），这样命名是因为这里会存在线程竞争。这种情况出现的机会往往非常小——就是因为竞争引起的错误非常不易重现，所以调试这种错误才会非常困难。避免并发和防止竞争条件称为同步（synchronization）。

① 术语执行线程（thread execution）指任何正在执行的代码实例，比如，一个在内核执行的进程、一个中断处理程序或一个内核线程。在本章中将执行线程简称为线程，记住，这个术语指代的是任何正在执行的代码。

9.1.1 为什么我们需要保护

为了认清同步的必要性，我们首先要明白临界区无处不在。作为第一个例子，让我们考察一个现实世界的情况：ATM（自动柜员机，或叫自动提款机）。

自动提款机所进行的主要操作就是从个人银行账户取钱。某人走到机器前，插入 ATM 卡，输入密码作为验证，选择取款，输入金额，敲确认，取出钱，然后发信息通知本人。

当用户要求取某一特定的金额后，提款机需要确保在其账户上的确有那么多钱。如果有，取款机就要从现有的总额中扣除取款额。实现这一描述的代码如下：

```
int total = get_total_from_account();      /* 账户上的总额 */
int withdrawal = get_withdrawal_amount(); /* 用户要求的取款额 */

/* 检查用户账户上是否有足够的金额 */
if (total < withdrawal) {
    error("You do not have that much money!")
    return -1;
}

/* 好啦，用户有足够的金额，从总额中扣除取款额 */
total -= withdrawal;
update_total_funds(total);

/* 把钱给用户 */
spit_out_money(withdrawal);
```

现在让我们假定在用户账户上的另一个扣款操作同时发生。看看扣款是如何同时发生的：假定用户的配偶在另一台 ATM 上开始另外的取款；而这和上述扣款同时进行——或者以电子形式从账户转出资金，或者是银行从账户上扣除某一费用（因为现在的银行总是这么干），或者是其他任何形式的扣款。

正在取款的两个系统都会执行我们刚刚看到的类似的代码：首先检查扣款是否可能，然后计算新的总额，最后进行实际的扣款。现在让我们虚拟一些数字。假定第一次从 ATM 扣款额是 \$100，第二次扣除银行申请费 \$10——因为客户走入了银行。假定客户在银行总共有 \$105。显然，如果账户不出现赤字，这两个操作中有一个就无法完成。

你可能希望发生的顺序是这样的：收费事务先发生。\$10 小于 \$105，因此，从 105 中减去 10 得到新的总额 95，这 \$10 就装在银行的口袋里。之后，ATM 取款发生，但未取到，因为 \$95 小于 \$100。

在竞争的环境下，实际生活可能更有趣。假定两个事务几乎同时开始。两个事务都验证是否有足够的金额存在：\$105 既大于 \$100，也大于 \$10，所以，两个条件都满足。于是，取款过程从 \$105 减去 \$100，剩余 \$5。收费事务也如法炮制，从 \$105 减去 \$10，剩余 \$95。此刻，收费事务也更新新的总额，结果得到 \$95。这可是余额呀！

显而易见，金融机构必须确保类似情况决不发生。他们必须在某些操作期间对账户加锁，确保每个事务相对其他任何事务的操作是原子性的。这样的事务必须完整地发生，要么干脆不发生，但是决不能打断。

9.1.2 单个变量

现在，让我们看一个特殊计算的例子。考虑一个非常简单的共享资源：一个全局整型变量和一个简单的临界区，其中的操作仅仅是将整型变量的值增加 1。

```
i++;
```

该操作可以转化成类似于下面动作的机器指令序列：

```
得到当前变量 i 的值并且拷贝到一个寄存器中
将寄存器中的值加 1
把 i 的新值写回到内存中
```

现在假定有两个执行线程同时进入这个临界区，如果 *i* 的初始值是 7，那么，我们所期望的结果应该像下面这样（每一行代表一个时间单元）：

线程 1	线程 2
获得 i (7)	—
增加 i (7->8)	—
写回 i (8)	—
—	获得 i (8)
—	增加 i (8->9)
—	写回 i (9)

正如所期望的，7 被两个线程分别加 1 变为 9。但是，实际的执行序列却可能如下：

线程 1	线程 2
获得 i (7)	获得 i (7)
增加 i (7->8)	—
—	增加 i (7->8)
写回 i (8)	—
—	写回 i (8)

如果两个执行线程都在变量 *i* 值增加前读取它的初值，进而又分别增加变量 *i* 的值，最后再保存该值，那么变量 *i* 的值就变成了 8，而变量 *i* 的值本该是 9 的。这是最简单的临界区例子，幸好对于这种简单竞争条件的解决方法也同样简单——我们仅仅需要将这些指令作为一个不可分割的整体来执行就万事大吉了。多数处理器都提供了指令来原子地读变量、增加变量，然后再写回变量，使用这样的指令就能解决一些问题。使用这个原子指令唯一可能的结果是：

线程 1	线程 2
增加 i (7->8)	—
—	增加 i (8->9)
或者是相反：	
线程 1	线程 2
—	增加 i (7->8)
增加 i (8->9)	—

两个原子操作交错执行根本就不可能发生，因为处理器会从物理上确保这种不可能。使用这样的指令会缓解这种问题，内核也提供了一组实现这些原子操作的接口，我们将在第 10 章讨论它们。

9.2 加锁

现在我们来讨论一个更为复杂的竞争条件，相应的解决方法也更为复杂。假设需要处理一个队列上的所有请求。我们假定该队列是通过链表得以实现，链表中的每个结点就代表一个请求。有两个函数可以用来操作此队列：一个函数将新请求添加到队列尾部，另一个函数从队列头删除请求，然后处理它。内核各个部分都会调用这两个函数，所以内核会不断地在队列中加入请求，从队列中删除和处理请求。对请求队列的操作无疑要用到多条指令。如果一个线程试图读取队列，而这时正好另一个线程正在处理该队列，那么读取线程就会发现队列此刻正处于不一致状态。很明显，如果允许并发访问队列，就会产生危害。当共享资源是一个复杂的数据结构时，竞争条件往往会使该数据结构遭到破坏。

表面上看，这种情况好像没有一个好的方法来解决，一个处理器读取队列的时候，我们怎么能禁止另一个处理器更新队列呢？虽然有些体系结构提供了简单的原子指令，实现算术运算和比较之类的原子操作，但让体系结构提供专门的指令，对像上例中那样的不定长度的临界区进行保护，就强人所难了。我们需要一种方法确保一次有且只有一个线程对数据结构进行操作，或者当另一个线程在对临界区标记时，就禁止（或者说锁定）其他访问。

锁提供的就是这种机制：它就如同一把门锁，门后的房间可想象成一个临界区。在一个指定时间内，房间里只能有一个执行线程存在，当一个线程进入房间后，它会锁住身后的房门；当它结束对共享数据的操作后，就会走出房间，打开门锁。如果另一个线程在房门上锁时来了，那么它就必须等待房间内的线程出来并打开门锁后，才能进入房间。这样，线程持有锁，而锁保护了数据。

前面例子中讲到的请求队列，可以使用一个单独的锁进行保护。每当有一个新请求要加入队列，线程会首先占住锁，然后就可以安全地将请求加入到队列中，结束操作后再释放该锁；同样当一个线程想从请求队列中删除一个请求时，也需要先占住锁，然后才能从队列中读取和删除请求，而且在完成操作后也必须释放锁。任何要访问队列的其他线程也类似，必须占住锁后才能进行操作。因为在一个时刻只能有一个线程持有锁，所以在一个时刻只有一个线程可以操作队列。如果一个线程正在更新队列时，另一个线程出现了，则第二个线程必须等待第一个线程释放锁，它才能继续进行。由此可见锁机制可以防止并发执行，并且保护队列不受竞争条件的影响。

任何要访问队列的代码首先都需要占住相应的锁，这样该锁就能阻止来自其他执行线程的并发访问：

线程 1

试图锁定队列
成功：获得锁
访问队列…
为队列解除锁
…

线程 2

试图锁定队列
失败：等待…
等待…
等待…
成功：获得锁
访问队列…
为队列解除锁

请注意锁的使用是自愿的、非强制的，它完全属于一种编程者自选的编程手段。没有什么可以强制编程者在操作我们虚构的队列时必须使用锁。当然，如果不这么做，无疑会造成竞争条件

而破坏队列。

锁有多种多样的形式，而且加锁的粒度范围也各不相同——Linux 自身实现了几种不同的锁机制。各种锁机制之间的区别主要在于：当锁已经被其他线程持有，因而不可用时行为表现——一些锁被争用时会简单地执行忙等待[⊖]，而另外一些锁会使当前任务睡眠直到锁可用为止。第 10 章我们将讨论 Linux 中不同锁之间的行为差别及它们的接口。

机灵的读者此时会尖叫起来，锁根本解决不了什么问题，它只不过是把临界区缩小到加锁和开锁之间（也许更小）的代码，但是仍然有潜在的竞争！所幸，锁是采用原子操作实现的，而原子操作不存在竞争。单一指令可以验证它的关键部分是否抓住，如果没有的话，就抓住它。其实现是与具体的体系结构密切相关的，但是，几乎所有的处理器都实现了测试和设置指令，这一指令测试整数的值，如果其值为 0，就设置一新值。0 意味着开锁。在流行的 x86 体系结构中，锁的实现也不例外，它使用了称为 compare 和 exchange 的类似指令。

9.2.1 造成并发执行的原因

用户空间之所以需要同步，是因为用户程序会被调度程序抢占和重新调度。由于用户进程可能在任何时刻被抢占，而调度程序完全可能选择另一个高优先级的进程到处理器上执行，所以就会使得一个程序正处于临界区时，被非自愿地抢占了。如果新调度的进程随后也进入同一个临界区（比如说，这两个进程要操作共享的内存，或者向同一个文件描述符中写入），前后两个进程相互之间就会产生竞争。另外，因为信号处理是异步发生的，所以，即使是单线程的多个进程共享文件[Ⓢ]，或者在一个程序内部处理信号，也有可能产生竞争条件。这种类型的并发操作——这里其实两者并不真是同时发生的，但它们相互交叉进行，所以也可称作伪并发执行。

如果你有一台支持对称多处理器的机器，那么两个进程就可以真正地在临界区中同时执行了，这种类型称为真并发。虽然真并发和伪并发的原因和含义不同，但它们都同样会造成竞争条件，而且也需要同样的保护。

内核中有类似可能造成并发执行的原因。它们是：

- 中断——中断几乎可以在任何时刻异步发生，也就可能随时打断当前正在执行的代码。
- 软中断和 tasklet——内核能在任何时刻唤醒或调度软中断和 tasklet，打断当前正在执行的代码。
- 内核抢占——因为内核具有抢占性，所以内核中的任务可能会被另一任务抢占。
- 睡眠及与用户空间的同步——在内核执行的进程可能会睡眠，这就会唤醒调度程序，从而导致调度一个新的用户进程执行。
- 对称多处理——两个或多个处理器可以同时执行代码。

对内核开发者来说，必须理解上述这些并发执行的原因，并且为它们事先做足准备工作。如果在一段内核代码操作某资源的时候系统产生了一个中断，而且该中断的处理程序还要访问这一资源，这就是一个 bug；类似地，如果一段内核代码在访问一个共享资源期间可以被抢占，这也是一个 bug；还有，如果内核代码在临界区里睡眠，那简直就是鼓掌欢迎竞争条件的到来。最后

[⊖] 也就是说，反复处于一个循环中，不断检测锁状态，等待锁变为可用。

还要注意，两个处理器绝对不能同时访问同一共享数据。当我们清楚什么样的数据需要保护时，提供锁来保护系统稳定也就不难做到了。然而，真正困难的就是发现上述的潜在并发执行的可能，并有意识地采取某些措施来防止并发执行。

我们要重申这点，因为它实在是很重要。其实，真正用锁来保护共享资源并不困难，尤其是在设计代码的早期就这么做了，事情就更简单了。辨认出真正需要共享的数据和相应的临界区，才是真正有挑战性的地方。要记住，最开始设计代码的时候就要考虑加入锁，而不是事后才想到。如果代码已经写好了，再在其中找到需要上锁的部分并向其中追加锁，是非常困难的，结果也往往不尽如人意。所以，避免这种亡羊补牢的做法是：在编写代码的开始阶段就要设计恰当的锁。

在中断处理程序中能避免并发访问的安全代码称作中断安全代码（interrupt-safe），在对称多处理的机器中能避免并发访问的安全代码称为 SMP 安全代码（SMP-safe），在内核抢占时能避免并发访问的安全代码称为抢占安全代码[⊖]（preempt-safe）。在第 10 章会重点讲述为了提供同步和避免所有上述竞争条件，内核所使用的实际方法。

9.2.2 了解要保护些什么

找出哪些数据需要保护是关键所在。由于任何可能被并发访问的代码都几乎无例外地需要保护，所以寻找哪些代码不需要保护反而相对更容易些，我们也就从这里入手。执行线程的局部数据仅仅被它本身访问，显然不需要保护，比如，局部自动变量（还有动态分配的数据结构，其地址仅存放在堆栈中）不需要任何形式的锁，因为它们独立存在于执行线程的栈中。类似的，如果数据只会被特定的进程访问，那么也不需要加锁（因为进程一次只在一个处理器上执行）。

到底什么数据需要加锁呢？大多数内核数据结构都需要加锁！有一条很好的经验可以帮助我们判断：如果有其他执行线程可以访问这些数据，那么就给这些数据加上某种形式的锁；如果任何其他什么东西都能看到它，那么就要锁住它。记住：要给数据而不是给代码加锁。

配置选项：SMP 与 UP

因为 Linux 内核可在编译时配置，所以你可以针对指定机器进行内核裁剪。更重要的是，CONFIG_SMP 配置选项控制内核是否支持 SMP。许多加锁问题在单处理器上是不存在的，因而当 CONFIG_SMP 没被设置时，不必要的代码就不会被编入针对单处理器的内核映像中。这样做可以使单处理器机器避免使用自旋锁带来的开销。同样的技巧也适用于 CONFIG_PREEMPT（允许内核抢占的配置选项）。这种设计真的很优越——内核只用维护一些简洁的基础资源，各种各样的锁机制当需要时可随时被编译进内核使用。在不同的体系结构上，CONFIG_SMP 和 CONFIG_PREEMPT 设置不同，实际编译时包含的锁就不同。

在代码中，要为大多数糟糕的情况提供适当的保护，例如具有内核抢占的 SMP，并且要考虑到所有的情况。

[⊖] 你将看到，除了一些意外情况外，SMP 安全其实也意味着抢占安全。

在编写内核代码时，你要问自己下面这些问题：

- 这个数据是不是全局的？除了当前线程外，其他线程能不能访问它？
- 这个数据会不会在进程上下文和中断上下文中共享？它是不是要在两个不同的中断处理程序中共享？
- 进程在访问数据时可不可能被抢占？被调度的新程序会不会访问同一数据？
- 当前进程是不是会睡眠（阻塞）在某些资源上，如果是，它会让共享数据处于何种状态？
- 怎样防止数据失控？
- 如果这个函数又在另一个处理器上被调度将会发生什么呢？
- 如何确保代码远离并发威胁呢？

简而言之，几乎访问所有的内核全局变量和共享数据都需要某种形式的同步方法，具体加锁方法将在第 10 章进行讨论。

9.3 死锁

死锁的产生需要一定条件：要有一个或多个执行线程和一个或多个资源，每个线程都在等待其中的一个资源，但所有的资源都已经被占用了。所有线程都在相互等待，但它们永远不会释放已经占有的资源。于是任何线程都无法继续，这便意味着死锁的发生。

一个很好的死锁例子是四路交通堵塞问题。如果每一个停止的车都决心等待其他的车开动后自己再启动，那么就没有任何一辆车能启动，于是就造成了交通死锁的发生。

最简单的死锁例子是自死锁^①：如果一个执行线程试图去获得一个自己已经持有的锁，它将不得不等待锁被释放，但因为它正在忙着等待这个锁，所以自己永远也不会有机会释放锁，最终结果就是死锁：

```
获得锁
再次试图获得锁
等待锁重新可用
.....
```

同样道理，考虑有 n 个线程和 n 个锁，如果每个线程都持有一把其他进程需要得到的锁，那么所有的线程都将阻塞地等待它们希望得到的锁重新可用。最常见的例子是有两个线程和两把锁，它们通常被叫做 ABBA 死锁。

线程 1

```
获得锁 A
试图获得锁 B
等待锁 B
```

线程 2

```
获得锁 B
试图获得锁 A
等待锁 A
```

每个线程都在等待其他线程持有的锁，但是绝没有一个线程会释放它们一开始就持有的锁，所以没有任何锁会在释放后被其他线程使用。

① 有些内核提供递归锁来防止自死锁现象，递归锁可以被一个执行线程多次请求。幸好 Linux 没有提供这样的递归锁。不用递归锁通常被认为是一件好事，虽然递归锁缓和了自死锁问题，但它们很容易使加锁逻辑变得杂乱无章。

预防死锁的发生非常重要，虽然很难证明代码不会发生死锁，但是可以写出避免死锁的代码，一些简单的规则对避免死锁大有帮助：

- 按顺序加锁。使用嵌套的锁时必须保证以相同的顺序获取锁，这样可以阻止致命拥抱类型的死锁。最好能记录下锁的顺序，以便其他人也能照此顺序使用。
- 防止发生饥饿。试问，这个代码的执行是否一定会结束？如果“张”不发生？“王”要一直等待下去吗？
- 不要重复请求同一个锁。
- 设计应力求简单——越复杂的加锁方案越有可能造成死锁。

最值得强调的是第一点，它最为重要。如果有两个或多个锁曾在同一时间里被请求，那么以后其他函数请求它们也必须按照前次的加锁顺序进行。假设有 cat、dog 和 fox 这几个锁来保护某同名的多个数据结构，同时假设有一个函数对这三个锁保护的数据结构进行操作——可能在它们之间进行拷贝。不管哪种情况，这些数据结构都需要保护才能被安全访问。如果有一个函数以 cat、dog，然后是 fox 的顺序获得了锁，那么其他任何函数都必须以同样的顺序来获得这些锁（或是它们的子集）。如果其他函数首先获得锁 fox，然后获得锁 dog（因为锁 dog 总是应该先于锁 fox 被获得），就有发生死锁的可能（所以是个 bug）。为更直观地说明，下面给出一个造成死锁的例子：

线程 1

```
获得锁 cat
获得锁 dog
试图获得锁 fox
等待锁 fox
```

线程 2

```
获得锁 fox
试图获得锁 dog
等待锁 dog
.....
```

线程 1 在等待锁 fox，而该锁此刻被线程 2 持有；同样线程 2 正在等待锁 dog，而该锁此刻又被线程 1 持有。任何一方都不会放弃自己已持有的锁，于是双方都会永远地等待下去——也就是死锁。但是，只要线程都按照相同的顺序去获取这些锁，就可以避免上述的死锁情况。

只要嵌套地使用多个锁，就必须按照相同的顺序去获取它们。在代码中使用锁的地方，对锁的获取顺序加上注释是个良好的习惯。下面的例子就做得很不错：

```
/*
 * cat_lock——用于保护访问 cat 数据结构的锁，总是要在获得锁 dog 前先获得
 */
```

尽管释放锁的顺序和死锁是无关的，但最好还是以获得锁的相反顺序来释放锁。

防止死锁很重要，所以 Linux 内核提供了一些简单易用的调试工具，可以在运行时检测死锁，我们将在第 10 章讨论它们。

9.4 争用和扩展性

锁的争用（lock contention），或简称争用，是指当锁正在被占用时，有其他线程试图获得该锁。说一个锁处于高度争用状态，就是指有多个其他线程在等待获得该锁。由于锁的作用是使程序以串行方式对资源进行访问，所以使用锁无疑会降低系统的性能。被高度争用（频繁被持有，

或者长时间持有——两者都有就更糟糕)的锁会成为系统的瓶颈,严重降低系统性能。即使是这样,相比于被几个相互抢夺共享资源的线程撕成碎片,搞得内核崩溃,还是这种同步保护来得更好一点。当然,如果有办法能解决高度争用问题,就更好不过了。

扩展性(scalability)是对系统可扩展程度的一个量度。对于操作系统,我们在谈及可扩展性时就会和大量进程、大量处理器或是大量内存等联系起来。其实任何可以被计量的计算机组件都可以涉及可扩展性。理想情况下,处理器的数量加倍应该会使系统处理性能翻倍。而实际上,这是不可能达到的。

自从 2.0 版内核引入多处理支持后, Linux 对集群处理器的可扩展性大大提高了。在 Linux 刚加入对多处理器支持的时候,一个时刻只能有一个任务在内核中执行;在 2.2 版本中,当加锁机制发展到细粒度(fine-grained)加锁后,便取消了这种限制,而在 2.4 和后续版本中,内核加锁的粒度变得越来越精细。如今,在 Linux 2.6 版内核中,内核加的锁是非常细的粒度,可扩展性也很好。

加锁粒度用来描述加锁保护的数据规模。一个过粗的锁保护大块数据——比如,一个子系统用到的所有的数据结构;相反,一个过于精细的锁保护很小的一块数据——比如,一个大数据结构中的一个元素。在实际使用中,绝大多数锁的加锁范围都处于上述两种极端之间,保护的既不是一个完整的子系统也不是一个独立元素,而可能是一个单独的数据结构。许多锁的设计在开始阶段都很粗,但是当锁的争用问题变得严重时,设计就向更加精细的加锁方向进化。

在第 4 章中讨论过的运行队列,就是一个锁从粗到精细化的实例。在 2.4 版和更早的内核中,调度程序有一个单独的调度队列(回忆一下,调度队列是一个由可调度进程组成的链表),在 2.6 版内核系列的早期版本中, $O(1)$ 调度程序为每个处理器单独配备一个运行队列,每个队列拥有自己的锁,于是加锁由一个全局锁精化到了每个处理器拥有各自的锁。这是一种重要的优化,因为运行队列锁在大型机器上被争着用,本质上就是要在调度程序中每次都把整个调度进程下放到单个处理器上执行。在 2.6 版内核系列的新近版本中, CFS 调度器进一步提升了锁的可扩展性。

一般来说,提高可扩展性是件好事,因为它可以提高 Linux 在更大型的、处理能力更强大的系统上的性能。但是一味地“提高”可扩展性,却会导致 Linux 在小型 SMP 和 UP 机器上的性能降低,这是因为小型机器可能用不到特别精细的锁,锁得过细只会增加复杂度,并加大开销。考虑一个链表,最初的加锁方案可能就是用锁来保护链表,后来发现,在拥有集群处理器机器上,当各个处理器需要频繁访问该链表的时候,只用单独一个锁却成了扩展性的瓶颈。为解决这个瓶颈,我们将原来加锁的整个链表变成为链表中的每一个结点都加入自己的锁,这样一来,如果要对结点进行读写,必须先得到这个结点对应的锁。将加锁粒度变细后,多处理器访问同一个结点时,只会争用一个锁。可是这时锁的争用仍然没有完全避免,那么,能不能为每个结点中的每个元素都提供一个锁呢?(答案是:不能。)严格地讲,即使这么细的锁可以在大规模 SMP 机器上执行得很好,但它在双处理器机器上的表现又会怎样呢?如果在双处理器机器锁争用表现得并不明显,那么多余的锁会加大系统开销,造成很大的浪费。

不管怎么说,可扩展性都是很重要的,需要慎重考虑。关键在于,在设计锁的开始阶段就应该考虑到要保证良好的扩展性。因为即使在小型机器上,如果对重要资源锁得太粗,也很容易造成系统性能瓶颈。锁加得过粗或过细,差别往往只在一念之间。当锁争用严重时,加锁太粗会降

低可扩展性；而锁争用不明显时，加锁过细会加大系统开销，带来浪费，这两种情况都会造成系统性能下降。但要记住：设计初期加锁方案应该力求简单，仅当需要时再进一步细化加锁方案。精髓在于力求简单。

9.5 小结

要编写 SMP 安全代码，不能等到编码完成后才考虑如何加锁。恰当的同步（也就是加锁）（既要满足不死锁、可扩展，而且还要清晰、简洁）需要从头到尾，在整个编码过程中不断考虑与完善。无论你在编写哪种内核代码，是新的系统调用也好，还是重写驱动程序也好，首先应该考虑的就是保护数据不被并发访问，记住，加锁你的代码。

第 10 章将讨论如何为 SMP、内核抢占和其他各种情况提供充分的同步保护，确保数据在任何机器和配置中的安全。

了解了同步、并发和加锁的基本原理之后，让我们现在潜心钻研 Linux 内核提供的实际工具，以确保你的代码有竞争力但免于死锁。



第 10 章

内核同步方法

第 9 章讨论了竞争条件为何会产生以及怎么去解决。幸运的是，Linux 内核提供了一组相当完备的同步方法，这些方法使得内核开发者们能编写出高效而又自由竞争的代码。本章讨论的就是这些方法，包括它们的接口、行为和用途。

10.1 原子操作

我们首先介绍同步方法中的原子操作，因为它是其他同步方法的基石。原子操作可以保证指令以原子的方式执行——执行过程不被打断。众所周知，原子原本指的是不可分割的微粒，所以原子操作也就是不能够被分割的指令。例如，第 9 章曾提到过的原子方式的加操作，它通过把读取和增加变量的行为包含在一个单步中执行，从而防止了竞争的发生保证了操作结果总是一致的。一起来回忆一下这个整数递加过程中遇到的竞争吧：

线程 1

获得 i (7)
增加 i (7->8)
—
写回 i (8)
└

线程 2

获得 i (7)

增加 i (8->9)
—
写回 i (8)

使用原子操作，上述的竞争不会发生——事实上不可能发生。从而，计算过程无疑会是下述之一：

线程 1

获得、增加和存储 i (7 -> 8)
—

线程 2

—
获得、增加和存储 i (8 -> 9)

或者是

线程 1

—
获得、增加和存储 i (8 -> 9)

线程 2

获得、增加和存储 i (7 -> 8)
—

最后得到的 9，毫无疑问是正确结果。两个原子操作绝对不可能并发地访问同一个变量，这样加操作也就绝不可能引起竞争。

内核提供了两组原子操作接口——一组针对整数进行操作，另一组针对单独的位进行操作。在 Linux 支持的所有体系结构上都实现了这两组接口。大多数体系结构会提供支持原子操作的简单算术指令。而有些体系结构确实缺少简单的原子操作指令，但是也为单步执行提供了锁内存总线的指令，这就确保了其他改变内存的操作不能同时发生。

10.1.1 原子整数操作

针对整数的原子操作只能对 `atomic_t` 类型的数据进行处理。在这里之所以引入了一个特殊数据类型，而没有直接使用 C 语言的 `int` 类型，主要是出于两个原因：首先，让原子函数只接收 `atomic_t` 类型的操作数，可以确保原子操作只与这种特殊类型数据一起使用。同时，这也保证了该类型的数据不会被传递给任何非原子函数。实际上，对一个数据一会儿要采用原子操作，一会儿又不用原子操作了，这又能有什么好处？其次，使用 `atomic_t` 类型确保编译器不对（不能说完美地完成了任务但不乏自知之明）相应的值进行访问优化——这点使得原子操作最终接收到正确的内存地址，而不只是一个别名。最后，在不同体系结构上实现原子操作的时候，使用 `atomic_t` 可以屏蔽其间的差异。`Atomic_t` 类型定义在文件 `<linux/types.h>` 中：

```
typedef struct {
    volatile int counter;
} atomic_t;
```

尽管 Linux 支持的所有机器上的整型数据都是 32 位的，但是使用 `atomic_t` 的代码只能将该类型的数据当做 24 位来用。这个限制完全是因为在 SPARC 体系结构上，原子操作的实现不同于其他体系结构：32 位 `int` 类型的低 8 位被嵌入了一个锁（如图 10-1 所示），因为 SPARC 体系结构对原子操作缺乏指令级的支持，所以只能利用该锁来避免对原子类型数据的并发访问。所以在 SPARC 机器上就只能使用 24 位了。虽然其他机器上的代码完全可以使用全部的 32 位，但在 SPARC 机器上却可能造成一些奇怪和微妙的错误——这简直太不和谐了。最近，机灵的黑客已经允许 SPARC 提供全 32 位的 `atomic_t`，这一限制不存在了。

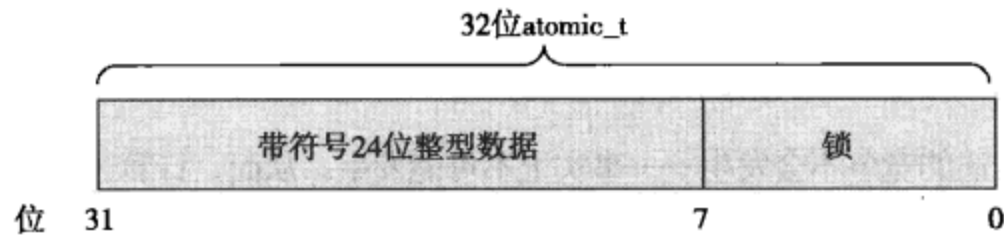


图 10-1 SPARC 上的 32 位 `atomic_t` 的布局

使用原子整型操作需要的声明都在 `<asm/atomic.h>` 文件中。有些体系结构会提供一些只能在该体系结构上使用的额外原子操作方法，但所有的体系结构都能保证内核使用到的所有操作的最小集。在写内核代码时，可以肯定，这个最小操作集在所有体系结构上都已实现了。

定义一个 `atomic_t` 类型的数据方法很平常，你还可以在定义时给它设定初值：

```
atomic_t v; /* 定义 v */
atomic_t u = ATOMIC_INIT(0); /* 定义 u 并把它初始化为 0 */
```

操作也都非常简单：

```
atomic_set(&v, 4); /* v = 4 (原子地) */
atomic_add(2, &v); /* v = v + 2 = 6 (原子地) */
atomic_inc(&v); /* v = v + 1 = 7 (原子地) */
```

如果需要将 `atomic_t` 转换成 `int` 型, 可以使用 `atomic_read()` 来完成:

```
printk("%d\n", atomic_read(&v)); /* 会打印 "7" */
```

原子整数操作最常见的用途就是实现计数器。使用复杂的锁机制来保护一个单纯的计数器显然杀鸡用了宰牛刀, 所以, 开发者最好使用 `atomic_inc()` 和 `atomic_dec()` 这两个相对来说轻便一点的操作。

还可以用原子整数操作原子地执行一个操作并检查结果。一个常见的例子就是原子地减操作和检查。

```
int atomic_dec_and_test(atomic_t *v)
```

这个函数将给定的原子变量减 1, 如果结果为 0, 就返回真; 否则返回假。表 10-1 列出了所有的标准原子整数操作 (所有体系结构都包含这些操作)。某种特定的体系结构上实现的所有操作可以在文件 `<asm/atomic.h>` 中找到。

表 10-1 原子整数操作列表

原子整数操作	描 述
<code>ATOMIC_INIT(int i)</code>	在声明一个 <code>atomic_t</code> 变量时, 将它初始化为 <code>i</code>
<code>int atomic_read(atomic_t *v)</code>	原子地读取整数变量 <code>v</code>
<code>void atomic_set(atomic_t *v, int i)</code>	原子地设置 <code>v</code> 值为 <code>i</code>
<code>void atomic_add(int i, atomic_t *v)</code>	原子地给 <code>v</code> 加 <code>i</code>
<code>void atomic_sub(int i, atomic_t *v)</code>	原子地从 <code>v</code> 减 <code>i</code>
<code>void atomic_inc(atomic_t *v)</code>	原子地给 <code>v</code> 加 1
<code>void atomic_dec(atomic_t *v)</code>	原子地从 <code>v</code> 减 1
<code>int atomic_sub_and_test(int i, atomic_t *v)</code>	原子地从 <code>v</code> 减 <code>i</code> , 如果结果等于 0, 返回真; 否则返回假
<code>int atomic_add_negative(int i, atomic_t *v)</code>	原子地给 <code>v</code> 加 <code>i</code> , 如果结果是负数, 返回真; 否则返回假
<code>int atomic_add_return(int i, atomic_t *v)</code>	原子地给 <code>v</code> 加 <code>i</code> , 且返回结果
<code>int atomic_sub_return(int i, atomic_t *v)</code>	原子地从 <code>v</code> 减 <code>i</code> , 且返回结果
<code>int atomic_inc_return(int i, atomic_t *v)</code>	原子地给 <code>v</code> 加 1, 且返回结果
<code>int atomic_dec_return(int i, atomic_t *v)</code>	原子地从 <code>v</code> 减 1, 且返回结果
<code>int atomic_dec_and_test(atomic_t *v)</code>	原子地从 <code>v</code> 减 1, 如果结果等于 0, 返回真; 否则返回假
<code>int atomic_inc_and_test(atomic_t *v)</code>	原子地给 <code>v</code> 加 1, 如果结果等于 0, 返回真; 否则返回假

原子操作通常是内联函数, 往往是通过内嵌汇编指令来实现的。如果某个函数本来就是原子的, 那么它往往会被定义成一个宏。例如, 在大部分体系结构上, 读取一个字本身就是一种原子操作, 也就是说, 在对一个字进行写入操作期间不可能完成对该字的读取。这样, 把 `atomic_read()` 定义成一个宏, 只须返回 `atomic_t` 类型的整数值就可以了。

```

/**
 * atomic_read - read atomic variable
 * @v: pointer of type atomic_t
 *
 * Atomically reads the value of @v.
 */
static inline int atomic_read(const atomic_t *v)
{
    return v->counter;
}

```

原子性与顺序性的比较

关于原子读取的上述讨论引发了原子性与顺序性之间差异的讨论。正如所讨论的，一个字长的读取总是原子地发生，绝不可能对同一个字交错地进行写；读总是返回一个完整的字，这或者发生在写操作之前，或者之后，绝不可能发生在写的过程中。例如，如果一个整数初始化为 42，然后又置为 365，那么读取这个整数肯定会返回 42 或者 365，而绝不会是二者的混合。这就是我们所谓的原子性。

也许代码比这有更多的要求。或许要求读必须在待定的写之前发生——这种需求其实不属于原子性要求，而是顺序要求。原子性确保指令执行期间不被打断，要么全部执行完，要么根本不执行。另一方面，顺序性确保即使两条或多条指令出现在独立的执行线程中，甚至独立的处理器上，它们本该的执行顺序却依然要保持。

在本小节讨论的原子操作只保证原子性。顺序性通过屏障（barrier）指令来实施，这将在本章的后面讨论。

在编写代码的时候，能使用原子操作时，就尽量不要使用复杂的加锁机制。对多数体系结构来讲，原子操作与更复杂的同步方法相比较，给系统带来的开销小，对高速缓存行（cache-line）的影响也小。但是，对于那些有高性能要求的代码，对多种同步方法进行测试比较，不失为一种明智的做法。

10.1.2 64 位原子操作

随着 64 位体系结构越来越普及，内核开发者确实在考虑原子变量除 32 位 `atomic_t` 类型外，也应引入 64 位的 `atomic64_t`。因为移植性原因，`atomic_t` 变量大小无法在体系结构之间改变。所以，`atomic_t` 类型即便在 64 位体系结构下也是 32 位的，若要使用 64 位的原子变量，则使用 `atomic64_t` 类型——其功能和其 32 位的兄弟无异，使用方法完全相同，不同的只有整型变量大小从 32 位变成了 64 位。几乎所有的经典 32 位原子操作都有 64 位的实现，它们被冠以 `atomic64` 前缀，而 32 位实现冠以 `atomic` 前缀。表 10-2，是所有标准原子操作列表；有些体系结构实现的方法更多，但是没有移植性。与 `atomic_t` 一样，`atomic64_t` 类型其实是对长整型的一个简单封装类。

```

typedef struct {
    volatile long counter;
} atomic64_t;

```

表 10-2 原子整型操作

原子整数操作	描 述
ATOMIC64_INIT(long i)	在声明一个 atomic_t 变量时, 将它初始化为 i
long atomic64_read(atomic64_t *v)	原子地读取整数变量 v
void atomic64_set(atomic64_t *v, int i)	原子地设置 v 值为 i
void atomic64_add(int i, atomic64_t *v)	原子地给 v 加 i
void atomic64_sub(int i, atomic64_t *v)	原子地从 v 减 i
void atomic64_inc(atomic64_t *v)	原子地给 v 加 1
void atomic64_dec(atomic64_t *v)	原子地从 v 减 1
int atomic64_sub_and_test(int i, atomic64_t *v)	原子地从 v 减 i, 如果结果等于 0, 返回真; 否则返回假
int atomic64_add_negative(int i, atomic64_t *v)	原子地给 v 加 i, 如果结果是负数, 返回真; 否则返回假
long atomic64_add_return(int i, atomic64_t *v)	原子地给 v 加 i, 且返回结果
long atomic64_sub_return(int i, atomic64_t *v)	原子地从 v 减 i, 且返回结果
long atomic64_inc_return(int i, atomic64_t *v)	原子地给 v 加 i, 且返回结果
long atomic64_dec_return(int i, atomic64_t *v)	原子地从 v 减 i, 且返回结果
int atomic64_dec_and_test(atomic64_t *v)	原子地从 v 减 i, 如果结果等于 0, 返回真; 否则返回假
int atomic64_inc_and_test(atomic64_t *v)	原子地给 v 加 i, 如果结果等于 0, 返回真; 否则返回假

所有 64 位体系结构都提供了 atomic64_t 类型, 以及一组对应的算数操作方法。但是多数 32 位体系机构不支持 atomic64_t 类型——不过, x86-32 是一个众所周知的例外。为了便于在 Linux 支持的各种体系结构之间移植代码, 开发者应该使用 32 位的 atomic_t 类型。把 64 位的 atomic64_t 类型留给那些特殊体系结构和需要 64 位的代码吧。

10.1.3 原子位操作

除了原子整数操作外, 内核也提供了一组针对位这一级数据进行操作的函数。没什么好奇怪的, 它们是与体系结构相关的操作, 定义在文件 <asm/bitops.h> 中。

令人感到奇怪的是位操作函数是对普通的内存地址进行操作的。它的参数是一个指针和一个位号, 第 0 位是给定地址的最低有效位。在 32 位机上, 第 31 位是给定地址的最高有效位而第 32 位是下一个字的最低有效位。虽然使用原子位操作在多数情况下是对一个字长的内存进行访问, 因而位号应该位于 0 ~ 31 (在 64 位机器中是 0 ~ 63), 但是, 对位号的范围并没有限制。

由于原子位操作是对普通的指针进行的操作, 所以不像原子整型对应 atomic_t, 这里没有特殊的数据类型。相反, 只要指针指向了任何你希望的数据, 你就可以对它进行操作。来看一个例子:

```

unsigned long word = 0;

set_bit(0,&word);      /* 第 0 位被设置 (原子地) */
set_bit(1,&word);      /* 第 1 位被设置 (原子地) */
printk("%u\n",word);  /* 打印 3 */
clear_bit(1,&word);    /* 清空第 1 位 */
change_bit(0,&word);   /* 翻转第 0 位的值, 这里它被清空 */

/* 原子地设置第 0 位并且返回设置前的值 (0) */
if(test_and_set_bit(0,&word){
    /* 永远不为真 */
}

/* 下面的语句是合法的; 你可以把原子位指令与一般的 C 语句混在一起 */
word = 7;

```

在表 10-3 中给出了标准原子位操作列表。

表 10-3 原子位操作的列表

原子位操作	描 述
<code>void set_bit(int nr,void *addr)</code>	原子地设置 <code>addr</code> 所指对象的第 <code>nr</code> 位
<code>void clear_bit(int nr,void *addr)</code>	原子地清空 <code>addr</code> 所指对象的第 <code>nr</code> 位
<code>void change_bit(int nr,void *addr)</code>	原子地翻转 <code>addr</code> 所指对象的第 <code>nr</code> 位
<code>int test_and_set_bit(int nr,void *addr)</code>	原子地设置 <code>addr</code> 所指对象的第 <code>nr</code> 位, 并返回原先的值
<code>int test_and_clear_bit(int nr,void *addr)</code>	原子地清空 <code>addr</code> 所指对象的第 <code>nr</code> 位, 并返回原先的值
<code>int test_and_change_bit(int nr,void *addr)</code>	原子地翻转 <code>addr</code> 所指对象的第 <code>nr</code> 位, 并返回原先的值
<code>int test_bit(int nr,void *addr)</code>	原子地返回 <code>addr</code> 所指对象的第 <code>nr</code> 位

为方便起见, 内核还提供了一组与上述操作对应的非原子位函数。非原子位函数与原子位函数的操作完全相同, 但是, 前者不保证原子性, 且其名字前缀多两个下划线。例如, 与 `test_bit()` 对应的非原子形式是 `__test_bit()`。如果你不需要原子性操作 (比如说, 你已经用锁保护了自己的数据), 那么这些非原子的位函数相比原子的位函数可能会执行得更快些。

非原子位操作到底是什么?

乍一看, 非原子位操作没有任何意义。因为仅仅涉及一个位, 所以不存在发生矛盾的可能。只要其中的一个操作成功, 还会有什么事? 的确, 顺序性可能是重要的, 但我们在此正谈论原子性。到了最后, 如果这一位有了任一条指令所设置的值, 我们应当友好地离开, 对吗?

让我们跳回到原子性看看到底意味着什么。原子性意味着, 或者指令完整地成功执行完, 不被打断, 或者根本不执行。所以, 如果你连续执行两个原子位操作, 你会希望两个操作都成功。在操作都完成后, 位的值应该是第二个操作所赋予的。但是, 在最后一个操作发生前的某个时间点, 位的值应该维持第一个操作所赋予的。换句话说, 真正的原子操作需要的是——所有中间结果都正确无误。

例如，假定给出两个原子位操作：先对某位置位，然后清 0。如果没有原子操作，那么，这一位可能的确清 0 了，但是也可能根本没有置位。置位操作可能与清除操作同时发生，但没有成功。清除操作可能成功了，这一位如愿呈现为清 0。但是，有了原子操作，置位会真正发生，可能有那么一刻，读操作显示所置的位，然后清除操作才执行，该位变为 0 了。

这种行为可能是重要的，尤其当顺序性开始起作用的时候，或者当操作硬件寄存器的时候。

内核还提供了两个例程用来从指定的地址开始搜索第一个被设置（或未被设置）的位。

```
int find_first_bit(unsigned long *addr, unsigned int size)
int find_first_zero_bit(unsigned long *addr, unsigned int size)
```

这两个函数中第一个参数是一个指针，第二个参数是要搜索的总位数，返回值分别是第一个被设置的（或没被设置的）位的位号。如果你的搜索范围仅限于一个字，使用 `_ffs()` 和 `ffz()` 这两个函数更好，它们只需要给定一个要搜索的地址做参数。

与原子整数操作不同，代码一般无法选择是否使用位操作，它们是唯一的、具有可移植性的设置特定位方法，需要选择的是使用原子位操作还是非原子位操作。如果你的代码本身已经避免了竞争条件，你可以使用非原子位操作，通常这样执行得更快，当然，这还要取决于具体的体系结构。

10.2 自旋锁

如果每个临界区都能像增加变量这样简单就好了，可惜现实总是残酷的。现实世界里，临界区甚至可以跨越多个函数。举个例子，我们经常会碰到这种情况：先得从一个数据结构中移出数据，对其进行格式转换和解析，最后再把它加入到另一个数据结构中。整个执行过程必须是原子的，在数据被更新完毕前，不能有其他代码读取这些数据。显然，简单的原子操作对此无能为力，这就需要使用更为复杂的同步方法——锁来提供保护。

Linux 内核中最常见的锁是自旋锁（spin lock）。自旋锁最多只能被一个可执行线程持有。如果一个执行线程试图获得一个被已经持有（即所谓的争用）的自旋锁，那么该线程就会一直进行忙循环—旋转—等待锁重新可用。要是锁未被争用，请求锁的执行线程便能立刻得到它，继续执行。在任意时间，自旋锁都可以防止多于一个的执行线程同时进入临界区。同一个锁可以用在多个位置，例如，对于给定数据的所有访问都可以得到保护和同步。

再回到第 9 章门和锁的例子，自旋锁相当于坐在门外等待同伴从里面出来，并把钥匙交给你。如果你到了门口，发现里面没有人，就可以抓到钥匙进入房间。如果你到了门口发现里面正好有人，就必须在门外等待钥匙，不断地检查房间是否为空。当房间为空时，你就可以抓到钥匙进入。正是因为有了钥匙（相当于自旋锁），才允许一次只有一个人（相当于执行线程）进入房间（相当于临界区）。

一个被争用的自旋锁使得请求它的线程在等待锁重新可用时自旋（特别浪费处理器时间），这种行为是自旋锁的要点。所以自旋锁不应该被长时间持有。事实上，这点正是使用自旋锁的初衷：在短期间内进行轻量级加锁。还可以采取另外的方式来处理对锁的争用：让请求线程睡眠，

直到锁重新可用时再唤醒它。这样处理器就不必循环等待，可以去执行其他代码。这也会带来一定的开销——这里有两次明显的上下文切换，被阻塞的线程要换出和换入，与实现自旋锁的少数几行代码相比，上下文切换当然有较多的代码。因此，持有自旋锁的时间最好小于完成两次上下文切换的耗时。当然我们大多数人都不会无聊到去测量上下文切换的耗时，所以我们让持有自旋锁的时间应尽可能的短就可以了^①。在下面内容中我们将讨论信号量，信号量便提供了上述第二种锁机制，它使得在发生争用时，等待的线程能投入睡眠，而不是旋转。

10.2.1 自旋锁方法

自旋锁的实现和体系结构密切相关，代码往往通过汇编实现。这些与体系结构相关的代码定义在文件 <asm/spinlock.h> 中，实际需要用到的接口定义在文件 <linux/spinlock.h> 中。自旋锁的基本使用形式如下：

```
DEFINE_SPINLOCK(mr_lock);
spin_lock(&mr_lock);
/* 临界区 ... */
spin_unlock(&mr_lock);
```

因为自旋锁在同一时刻至多被一个执行线程持有，所以一个时刻只能有一个线程位于临界区内，这就为多处理器机器提供了防止并发访问所需的保护机制。注意在单处理器机器上，编译的时候并不会加入自旋锁。它仅仅被当做一个设置内核抢占机制是否被启用的开关。如果禁止内核抢占，那么在编译时自旋锁会被完全剔除出内核。

警告：自旋锁是不可递归的！

Linux 内核实现的自旋锁是不可递归的，这点不同于自旋锁在其他操作系统中的实现。所以如果你试图得到一个你正持有的锁，你必须自旋，等待你自己释放这个锁。但你处于自旋忙等待中，所以你永远没有机会释放锁，于是你被自己锁死了。千万小心自旋锁！

自旋锁可以使用在中断处理程序中（此处不能使用信号量，因为它们会导致睡眠）。在中断处理程序中使用自旋锁时，一定要在获取锁之前，首先禁止本地中断（在当前处理器上的中断请求），否则，中断处理程序就会打断正持有锁的内核代码，有可能会试图去争用这个已经被持有的自旋锁。这样一来，中断处理程序就会自旋，等待该锁重新可用，但是锁的持有者在这个中断处理程序执行完毕前不可能运行。这正是我们在前面的内容中提到的双重请求死锁。注意，需要关闭的只是当前处理器上的中断。如果中断发生在不同的处理器上，即使中断处理程序在同一锁上自旋，也不会妨碍锁的持有者（在不同处理器上）最终释放锁。

内核提供的禁止中断同时请求锁的接口，使用起来很方便，方法如下。

```
DEFINE_SPINLOCK(mr_lock);
unsigned long flags;

spin_lock_irqsave(&mr_lock, flags);
/* 临界区 ... */
```

^① 在现在的抢占式内核中，这点尤为重要。锁的持有时间等价于系统的调度等待时间。

```
spin_unlock_irqrestore(&mr_lock, flags);
```

函数 `spin_lock_irqsave()` 保存中断的当前状态，并禁止本地中断，然后再去获取指定的锁。反过来 `spin_unlock_irqrestore()` 对指定的锁解锁，然后让中断恢复到加锁前的状态。所以即使中断最初是被禁止的，代码也不会错误地激活它们，相反，会继续让它们禁止。注意，`flags` 变量看起来像是由数值传递的，这是因为这些锁函数有些部分是通过宏的方式实现的。

在单处理器系统上，虽然在编译时抛弃掉了锁机制，但在上面例子中仍需要关闭中断，以禁止中断处理程序访问共享数据。加锁和解锁分别可以禁止和允许内核抢占。

锁什么？

使用锁的时候一定要对症下药，要有针对性。要知道需要保护的是数据而不是代码。尽管本章的例子讲的都是保护临界区的重要性，但是真正保护的其实是临界区中的数据，而不是代码。

大原则：针对代码加锁会使得程序难以理解，并且容易引发竞争条件，正确的做法应该是对数据而不是代码加锁。

既然不是对代码加锁，那就一定要用特定的锁来保护自己的共享数据。例如，“`struct foo` 由 `loo_lock` 加锁”。无论你何时需要访问共享数据，一定要先保证数据是安全的。而保证数据安全往往就意味着在对数据进行操作前，首先占用恰当的锁，完成操作后再释放它。

如果你能确定中断在加锁前是激活的，那就不需要在解锁后恢复中断以前的状态了。你可以无条件地在解锁时激活中断。这时，使用 `spin_lock_irq()` 和 `spin_unlock_irq()` 会更好一些。

```
DEFINE_SPINLOCK(mr_lock);

spin_lock_irq(&mr_lock);
/* 关键节 */
spin_unlock_irq(&mr_lock);
```

由于内核变得庞大而复杂，因此，在内核的执行路线上，你很难搞清楚中断在当前调用点上到底是不是处于激活状态。也正因为如此，我们并不提倡使用 `spin_lock_irq()` 方法。如果你一定要使用它，那你应该确定中断原来就处于激活状态，否则当其他人期望中断处于未激活状态时却发现处于激活状态，可能会非常不开心。

调试自旋锁

配置选项 `CONFIG_DEBUG_SPINLOCK` 为使用自旋锁的代码加入了许多调试检测手段。例如，激活了该选项，内核就会检查是否使用了未初始化的锁，是否在还没加锁的时候就要对锁执行开锁操作。在测试代码时，总是应该激活这个选项。如果需要进一步全程调试锁，还应该打开 `CONFIG_DEBUG_LOCK_ALLOC` 选项。

10.2.2 其他针对自旋锁的操作

你可以使用 `spin_lock_init()` 方法来初始化动态创建的自旋锁（此时你只有一个指向

`spinlock_t` 类型的指针，没有它的实体)。

`spin_try_lock()` 试图获得某个特定的自旋锁，如果该锁已经被争用，那么该方法会立刻返回一个非 0 值，而不会自旋等待锁被释放；如果成功地获得了这个自旋锁，该函数返回 0。同理，`spin_is_locked()` 方法用于检查特定的锁当前是否已被占用，如果已被占用，返回非 0 值；否则返回 0。该方法只做判断，并不实际占用^①。

表 10-4 给出了标准的自旋锁操作的完整列表。

表 10-4 自旋锁方法列表

方 法	描 述
<code>spin_lock()</code>	获取指定的自旋锁
<code>spin_lock_irq()</code>	禁止本地中断并获取指定的锁
<code>spin_lock_irqsave()</code>	保存本地中断的当前状态，禁止本地中断，并获取指定的锁
<code>spin_unlock()</code>	释放指定的锁
<code>spin_unlock_irq()</code>	释放指定的锁，并激活本地中断
<code>spin_unlock_irqrestore()</code>	释放指定的锁，并让本地中断恢复到以前状态
<code>spin_lock_init()</code>	动态初始化指定的 <code>spinlock_t</code>
<code>spin_trylock()</code>	试图获取指定的锁，如果未获取，则返回非 0
<code>spin_is_locked()</code>	如果指定的锁当前正在被获取，则返回非 0，否则返回 0

10.2.3 自旋锁和下半部

在第 8 章中曾经提到，在与下半部配合使用时，必须小心地使用锁机制。函数 `spin_lock_bh()` 用于获取指定锁，同时它会禁止所有下半部的执行。相应的 `spin_unlock_bh()` 函数执行相反的操作。

由于下半部可以抢占进程上下文中的代码，所以当下半部和进程上下文共享数据时，必须对进程上下文中的共享数据进行保护，所以需要加锁的同时还要禁止下半部执行。同样，由于中断处理程序可以抢占下半部，所以如果中断处理程序和下半部共享数据，那么就必须在获取恰当的锁的同时还要禁止中断。

回忆一下，同类的 `tasklet` 不可能同时运行，所以对于同类 `tasklet` 中的共享数据不需要保护。但是当数据被两个不同种类的 `tasklet` 共享时，就需要在访问下半部中的数据前先获得一个普通的自旋锁。这里不需要禁止下半部，因为在同一个处理器上绝不会有 `tasklet` 相互强占的情况。

对于软中断，无论是否同种类型，如果数据被软中断共享，那么它必须得到锁的保护。这是因为，即使是同种类型的两个软中断也可以同时运行在一个系统的多个处理器上。但是，同一处理器上的一个软中断绝不会抢占另一个软中断，因此，根本没必要禁止下半部。

10.3 读 - 写自旋锁

有时，锁的用途可以明确地分为读取和写入两个场景。例如，对一个链表可能既要更新又要

① 这两个方法往往让代码变得令人费解，一般来说用不着经常检查自旋锁的——你的代码本身应该要么直接请求占用锁，要么应该在占用锁之后才能被调用。不过，它们还是有些合理用途，所以 Linux 内核也就提供了这样的接口。

检索。当更新（写入）链表时，不能有其他代码并发地写链表或从链表中读取数据，写操作要求完全互斥。另一方面，当对其检索（读取）链表时，只要其他程序不对链表进行写操作就行了。只要没有写操作，多个并发的读操作都是安全的。任务链表的存取模式（在第3章中讨论过）就非常类似于这种情况，它就是通过读－写自旋锁获得保护的。

当对某个数据结构的操作可以像这样被划分为读／写或者消费者／生产者两种类别时，类似读／写锁这样的机制就很有帮助了。为此，Linux 内核提供了专门的读－写自旋锁。这种自旋锁为读和写分别提供了不同的锁。一个或多个读任务可以并发地持有读者锁；相反，用于写的锁最多只能被一个写任务持有，而且此时不能有并发的读操作。有时把读／写锁叫做共享／排斥锁，或者并发／排斥锁，因为这种锁以共享（对读者而言）和排斥（对写者而言）的形式获得使用。

读／写自旋锁的使用方法类似于普通自旋锁，它们通过下面的方法初始化：

```
DEFINE_RWLOCK(mr_rwlock);
```

然后，在读者的代码分支中使用如下函数：

```
read_lock(&mr_rwlock);  
/* 临界区（只读）...*/  
read_unlock(&mr_rwlock);
```

最后，在写者的代码分支中使用如下函数：

```
write_lock(&mr_rwlock);  
/* 临界区（读写）...*/  
write_unlock(&mr_rwlock);
```

通常情况下，读锁和写锁会位于完全分割开的代码分支中，如上例所示。

注意，不能把一个读锁“升级”为写锁。比如考虑下面这段代码：

```
read_lock(&mr_rwlock);  
write_lock(&mr_rwlock);
```

执行上述两个函数将会带来死锁，因为写锁会不断自旋，等待所有的读者释放锁，其中也包括它自己。所以当确实需要写操作时，要在一开始就请求写锁。如果写和读不能清晰地分开的话，那么使用一般的自旋锁就行了，不要使用读－写自旋锁。

多个读者可以安全地获得同一个读锁，事实上，即使一个线程递归地获得同一读锁也是安全的。这个特性使得读－写自旋锁真正成为有用并且常用的优化手段。如果在中断处理程序中只有读操作而没有写操作，那么，就可以混合使用“中断禁止”锁，使用 `read_lock()` 而不是 `read_lock_irqsave()` 对读进行保护。不过，你还是需要用 `write_lock_irqsave()` 禁止有写操作的中断，否则，中断里的读操作就有可能锁死在写锁上^①。表 10-5 列出了针对读－写自旋锁的所有操作。

① 假如读者正在进行操作，包含写操作的中断发生了，由于读锁还没有全部被释放，所以写操作会自旋，而读操作只能在包含写操作的中断返回后才能继续，释放读锁，此时死锁就发生了。——译者注

表 10-5 读-写自旋锁方法列表

方 法	描 述
<code>read_lock()</code>	获得指定的读锁
<code>read_lock_irq()</code>	禁止本地中断并获得指定读锁
<code>read_lock_irqsave()</code>	存储本地中断的当前状态, 禁止本地中断并获得指定读锁
<code>read_unlock()</code>	释放指定的读锁
<code>read_unlock_irq()</code>	释放指定的读锁并激活本地中断
<code>read_unlock_irqrestore()</code>	释放指定的读锁并将本地中断恢复到指定的前状态
<code>write_lock()</code>	获得指定的写锁
<code>write_lock_irq()</code>	禁止本地中断并获得指定写锁
<code>write_lock_irqsave()</code>	存储本地中断的当前状态, 禁止本地中断并获得指定写锁
<code>write_unlock()</code>	释放指定的写锁
<code>write_unlock_irq()</code>	释放指定的写锁并激活本地中断
<code>write_unlock_irqrestore()</code>	释放指定的写锁并将本地中断恢复到指定的前状态
<code>write_trylock()</code>	试图获得指定的写锁; 如果写锁不可用, 返回非 0 值
<code>rwlock_init()</code>	初始化指定的 <code>rwlock_t</code>

在使用 Linux 读-写自旋锁时, 最后要考虑的一点是这种锁机制照顾读比照顾写要多一点。当读锁被持有时, 写操作为了互斥访问只能等待, 但是, 读者却可以继续成功地作占用锁。而自旋等待的写者在所有读者释放锁之前是无法获得锁的。所以, 大量读者必定会使挂起的写者处于饥饿状态, 在你自己设计锁时一定要记住这一点——有些时候这种行为是有益的, 有时则会带来灾难。

自旋锁提供了一种快速简单的锁实现方法。如果加锁时间不长并且代码不会睡眠 (比如中断处理程序), 利用自旋锁是最佳选择。如果加锁时间可能很长或者代码在持有锁时有可能睡眠, 那么最好使用信号量来完成加锁功能。

10.4 信号量

Linux 中的信号量是一种睡眠锁。如果有一个任务试图获得一个不可用 (已经被占用) 的信号量时, 信号量会将其推进一个等待队列, 然后让其睡眠。这时处理器能重获自由, 从而去执行其他代码。当持有的信号量可用 (被释放) 后, 处于等待队列中的那个任务将被唤醒, 并获得该信号量。

让我们再一次回到门和钥匙的例子。当某个人到了门前, 他抓取钥匙, 然后进入房间。最大的差异在于当另一个人到了门前, 但无法得到钥匙时会发生什么情况。在这种情况下, 这家伙不是在徘徊等待, 而是把自己的名字写在一个列表中, 然后打盹去了。当里面的人离开房间时, 就在门口查看一下列表。如果列表上有名字, 他就对第一个名字仔细检查, 并在胸部给他一拳, 叫醒他, 让他进入房间。在这种方式中, 钥匙 (相当于信号量) 继续确保一次只有一个人 (相当于执行线程) 进入房间 (相当于临界区)。这就比自旋锁提供了更好的处理器利用率, 因为没有把时间花费在忙等待上, 但是, 信号量比自旋锁有更大的开销。生活总是一分为二的。

我们可以从信号量的睡眠特性得出一些有意思的结论：

- 由于争用信号量的进程在等待锁重新变为可用时会睡眠，所以信号量适用于锁会被长时间持有的情况。
- 相反，锁被短时间持有时，使用信号量就不太适宜了。因为睡眠、维护等待队列以及唤醒所花费的开销可能比锁被占用的全部时间还要长。
- 由于执行线程在锁被争用时睡眠，所以只能在进程上下文中才能获取信号量锁，因为在中断上下文中是不能进行调度的。
- 你可以在持有信号量时去睡眠（当然你也可能并不需要睡眠），因为当其他进程试图获得同一信号量时不会因此而死锁（因为该进程也只是去睡眠而已，而你最终会继续执行的）。
- 在你占用信号量的同时不能占用自旋锁。因为在你等待信号量时可能会睡眠，而在持有自旋锁时是不允许睡眠的。

以上这些结论阐明了信号量和自旋锁在使用上的差异。在使用信号量的大多数时候，你的选择余地并不大。往往在需要和用户空间同步时，你的代码会需要睡眠，此时使用信号量是唯一的选择。由于不受睡眠的限制，使用信号量通常来说更加容易一些。如果需要在自旋锁和信号量中做选择，应该根据锁被持有的时间长短做判断。理想情况当然是所有的锁定操作都应该越短越好。但如果你用的是信号量，那么锁定的时间长一点也能够接受。另外，信号量不同于自旋锁，它不会禁止内核抢占，所以持有信号量的代码可以被抢占。这意味着信号量不会对调度的等待时间带来负面影响。

10.4.1 计数信号量和二值信号量

最后要讨论的是信号量的一个有用特性，它可以同时允许任意数量的锁持有者，而自旋锁在一个时刻最多允许一个任务持有它。信号量同时允许的持有者数量可以在声明信号量时指定。这个值称为使用者数量（usage count）或简单地叫数量（count）。通常情况下，信号量和自旋锁一样，在一个时刻仅允许有一个锁持有者。这时计数等于 1，这样的信号量被称为二值信号量（因为它或者由一个任务持有，或者根本没有任务持有它）或者称为互斥信号量（因为它强制进行互斥）。另一方面，初始化时也可以把数量设置为大于 1 的非 0 值。这种情况，信号量被称为计数信号量（counting semaphore），它允许在一个时刻至多有 count 个锁持有者。计数信号量不能用来进行强制互斥，因为它允许多个执行线程同时访问临界区。相反，这种信号量用来对特定代码加以限制，内核中使用它的机会不多。在使用信号量时，基本上用到的都是互斥信号量（计数等于 1 的信号量）。

信号量在 1968 年由 Edsger Wybe Dijkstra^①提出，此后它逐渐成为一种常用的锁机制。信号量支持两个原子操作 P() 和 V()，这两个名字来自荷兰语 Proberen 和 Vershogen。前者叫做测试操作（字面意思是探查），后者叫做增加操作。后来的系统把两种操作分别叫做 down() 和 up()，

① Dijkstra 博士（1930—2002）是计算机科学史上最为成功的科学家之一，他在操作系统设计、算法理论和信号量概念的创建等诸多领域做出了卓越的贡献。他生于荷兰的鹿特丹，曾在克萨斯大学任教 15 年。不过他恐怕对 Linux 中间夹杂的大量 GOTO 语句不太满意。

Linux 也遵从这种叫法。down() 操作通过对信号量计数减 1 来请求获得一个信号量。如果结果是 0 或大于 0，获得信号量锁，任务就可以进入临界区。如果结果是负数，任务会被放入等待队列，处理器执行其他任务。该函数如同一个动词，降低 (down) 一个信号量就等于获取该信号量。相反，当临界区中的操作完成后，up() 操作用来释放信号量，该操作也被称作是提升 (upping) 信号量，因为它会增加信号量的计数值。如果在该信号量上的等待队列不为空，那么处于队列中等待的任务在被唤醒的同时会获得该信号量。

10.4.2 创建和初始化信号量

信号量的实现是与体系结构相关的，具体实现定义在文件 <asm/semaphore.h> 中。struct semaphore 类型用来表示信号量。可以通过以下方式静态地声明信号量——其中 name 是信号量变量名，count 是信号量的使用数量：

```
struct semaphore name;  
sema_init(&name, count);
```

创建更为普通的互斥信号量可以使用以下快捷方式，不用说，name 仍然是互斥信号量的变量名：

```
static DECLARE_MUTEX(name);
```

更常见的情况是，信号量作为一个大数据结构的一部分动态创建。此时，只有指向该动态创建的信号量的间接指针，可以使用如下函数来对它进行初始化：

```
sema_init(sem, count);
```

sem 是指针，count 是信号量的使用者数量。

与前面类似，初始化一个动态创建的互斥信号量时使用如下函数：

```
init_MUTEX(sem);
```

我不明白为什么 “mutex” 在 init_MUTEX() 中是大写，或者为什么 “init” 在这个函数名中放在前面，而在 sema_init() 中放在后面。不知道在你读第 8 章时，有没有被这些不一致的命名方法打闷呢。

10.4.3 使用信号量

函数 down_interruptible() 试图获取指定的信号量，如果信号量不可用，它将把调用进程置成 TASK_INTERRUPTIBLE 状态——进入睡眠。回忆第 3 章的内容，这种进程状态意味着任务可以被信号唤醒，一般来说这是件好事。如果进程在等待获取信号量的时候接收到了信号，那么该进程就会被唤醒，而函数 down_interruptible() 会返回 -EINTR。另外一个函数 down() 会让进程在 TASK_UNINTERRUPTIBLE 状态下睡眠。你应该不希望这种情况发生，因为这样一来，进程在等待信号量的时候就不再响应信号了。因此，使用 down_interruptible() 比使用 down() 更为普遍（也更正确）。也许你会觉得这两个函数名字起得有点不恰当，的确，这些命名并不很理想。

使用 `down_trylock()` 函数，你可以尝试以堵塞方式来获取指定的信号量。在信号量已被占用时，它立刻返回非 0 值；否则，它返回 0，而且让你成功持有信号量锁。

要释放指定的信号量，需要调用 `up()` 函数。例如：

```
/* 定义并声明一个信号量，名字为 mr_sem，用于信号量计数 */
static DECLARE_MUTEX(mr_sem);

/* 试图获取信号量 ... */
if (down_interruptible(&mr_sem))
    /* 信号被接收，信号量还未获取 */
}

/* 临界区 ... */

/* 释放给定的信号量 */
up(&mr_sem);
```

表 10-6 给出了针对信号量的方法的完整列表。

表 10-6 信号量方法列表

方 法	描 述
<code>sema_init(struct semaphore *,int)</code>	以指定的计数值初始化动态创建的信号量
<code>init_MUTEX(struct semaphore *)</code>	以计数值 1 初始化动态创建的信号量
<code>init_MUTEX_LOCKED(struct semaphore *)</code>	以计数值 0 初始化动态创建的信号量 (初始为加锁状态)
<code>down_interruptible(struct semaphore *)</code>	以试图获得指定的信号量，如果信号量 已被争用，则进入可中断睡眠状态
<code>down(struct semaphore *)</code>	以试图获得指定的信号量，如果信号量 已被争用，则进入不可中断睡眠状态
<code>down_trylock(struct semaphore *)</code>	以试图获得指定的信号量，如果信号量 已被争用，则立刻返回非 0 值
<code>up(struct semaphore *)</code>	以释放指定的信号量，如果睡眠队列不空， 则唤醒其中一个任务

10.5 读 - 写信号量

与自旋锁一样，信号量也有区分读 - 写访问的可能。与读 - 写自旋锁和普通自旋锁之间的关系差不多，读 - 写信号量也要比普通信号量更具优势。

读 - 写信号量在内核中是由 `rw_semaphore` 结构表示的，定义在文件 `<linux/rwsem.h>` 中。通过以下语句可以创建静态声明的读 - 写信号量：

```
static DECLARE_RWSEM(name);
```

其中 `name` 是新信号量名。

动态创建的读 - 写信号量可以通过以下函数初始化：

```
init_rwsem(struct rw_semaphore *sem);
```

所有的读 - 写信号量都是互斥信号量——也就是说，它们的引用计数等于 1，虽然它们只对写者互斥，不对读者。只要没有写者，并发持有读锁的读者数不限。相反，只有唯一的写者（在没有读者时）可以获得写锁。所有读 - 写锁的睡眠都不会被信号打断，所以它只有一个版本的 `down()` 操作。例如：

```
static DECLARE_RWSEM(mr_rwsem);

/* 试图获取信号量用于读 ... */
down_read(&mr_rwsem);

/* 临界区（只读）... */

/* 释放信号量 */
up_read(&mr_rwsem);
/* ... */

/* 试图获取信号量用于写 ... */
down_write(&mr_rwsem);

/* 临界区（读和写）... */

/* 释放信号量 */
up_write(&mr_rwsem);
```

与标准信号量一样，读 - 写信号量也提供了 `down_read_trylock()` 和 `down_write_trylock()` 方法。这两个方法都需要一个指向读 - 写信号量的指针作为参数。如果成功获得了信号量锁，它们返回非 0 值；如果信号量锁被争用，则返回 0。要小心（不知道为什么要这样）这与普通信号量的情形完全相反。

读 - 写信号量相比读 - 写自旋锁多一种特有的操作：`downgrade_write()`。这个函数可以动态地将获取的写锁转换为读锁。

读 - 写信号量和读 - 写自旋锁一样，除非代码中的读和写可以明白无误地分割开来，否则最好不要使用它。再强调一次，读 - 写机制使用是有条件的，只有在你的代码可以自然地界定出读 - 写时才有价值。

10.6 互斥体

直到最近，内核中唯一允许睡眠的锁是信号量。多数用户使用信号量只使用计数 1，说白了是把其作为一个互斥的排他锁使用——好比允许睡眠的自旋锁。不幸的是，信号量用途更通用，没多少使用限制。这点使得信号量适合用于那些较复杂的、未明情况下的互斥访问，比如内核于用户空间复杂的交互行为。但这也意味着简单的锁定而使用信号量并不方便，并且信号量也缺乏强制的规则来行使任何形式的自动调试，即便受限的调试也不可能。为了找到一个更简单睡眠锁，内核开发者们引入了互斥体（mutex）。确实，这个名字容易和我们的习惯称呼混淆。所以这里我们澄清一下，“互斥体（mutex）”这个称谓所指的是任何可以睡眠的强制互斥锁，比如使用

计数是 1 的信号量。但在最新的 Linux 内核中，“互斥体 (mutex)” 这个称谓现在也用于一种实现互斥的特定睡眠锁。也就是说，互斥体是一种互斥信号。

mutex 在内核中对应数据结构 mutex，其行为和使用计数为 1 的信号量类似，但操作接口更简单，实现也更高效，而且使用限制更强。静态地定义 mutex，你需要做：

```
DEFINE_MUTEX(name);
```

动态初始化 mutex，你需要做：

```
mutex_init(&mutex);
```

对互斥锁锁定和解锁并不难：

```
mutex_lock(&mutex);
/* 临界区 */
mutex_unlock(&mutex);
```

看到了吧，它就是一个简化版的信号量，因为不再需要管理任何使用计数。

表 10-7 是基本的 mutex 操作列表。

表 10-7 Mutex 方法

方 法	描 述
mutex_lock(struct mutex *)	为指定的 mutex 上锁，如果锁不可用则睡眠
mutex_unlock(struct mutex *)	为指定的 mutex 解锁
mutex_trylock(struct mutex *)	试图获取指定的 mutex，如果成功则返回 1；否则锁被获取，返回值是 0
mutex_is_locked (struct mutex *)	如果锁已被争用，则返回 1；否则返回 0

mutex 的简洁性和高效性源自于相比使用信号量更多的受限性。它不同于信号量，因为 mutex 仅仅实现了 Dijkstra 设计初衷中的最基本的行为。因此 mutex 的使用场景相对而言更严格、更定向了。

- 任何时刻中只有一个任务可以持有 mutex，也就是说，mutex 的使用计数永远是 1。
- 给 mutex 上锁者必须负责给其再解锁——你不能在一个上下文中锁定一个 mutex，而在另一个上下文中给它解锁。这个限制使得 mutex 不适合内核同用户空间复杂的同步场景。最常使用的方式是：在同一上下文中上锁和解锁。
- 递归地上锁和解锁是不允许的。也就是说，你不能递归地持有同一个锁，同样你也不能再去解锁一个已经被解开的 mutex。
- 当持有一个 mutex 时，进程不可以退出。
- mutex 不能在中断或者下半部中使用，即使使用 mutex_trylock() 也不行。
- mutex 只能通过官方 API 管理：它只能使用上节中描述的方法初始化，不可被拷贝、手动初始化或者重复初始化。

也许 mutex 结构最有用的特色是：通过一个特殊的调试模式，内核可以采用编程方式检查和警告任何践踏其约束法则的不老实行为。当打开内核配置选项 CONFIG_DEBUG_MUTEXES 后，就会有多种检测来确保这些（还有别的一些）约束得以遵守。这些调试手段无疑能帮助你

其他 mutex 使用者们都能以规范式的、简单化的使用模式对其使用。

10.6.1 信号量和互斥体

互斥体和信号量很相似，内核中两者共存会令人混淆。所幸，它们的标准使用方式都有简单的规范：除非 mutex 的某个约束妨碍你使用，否则相比信号量要优先使用 mutex。当你写新代码时，只有碰到特殊场合（一般是很底层代码）才会需要使用信号量。因此建议首选 mutex。如果发现不能满足其约束条件，且没有其他别的选择时，再考虑选择信号量。

10.6.2 自旋锁和互斥体

了解何时使用自旋锁，何时使用互斥体（或信号量）对编写优良代码很重要，但是多数情况下，并不需要太多的考虑，因为在中断上下文中只能使用自旋锁，而在任务睡眠时只能使用互斥体。表 10-8 回顾了一下各种锁的需求情况。

表 10-8 使用什么：自旋锁与信号量的比较

需 求	建议的加锁方法
低开销加锁	优先使用自旋锁
短期锁定	优先使用自旋锁
长期加锁	优先使用互斥体
中断上下文中加锁	使用自旋锁
持有锁需要睡眠	使用互斥体

10.7 完成变量

如果在内核中一个任务需要发出信号通知另一任务发生了某个特定事件，利用完成变量（completion variable）是使两个任务得以同步的简单方法。如果一个任务要执行一些工作时，另一个任务就会在完成变量上等待。当这个任务完成工作后，会使用完成变量去唤醒在等待的任务。这听起来很像一个信号量，的确如此——思想是一样的。事实上，完成变量仅仅提供了代替信号量的一个简单的解决方法。例如，当子进程执行或者退出时，vfork() 系统调用使用完成变量唤醒父进程。

完成变量由结构 completion 表示，定义在 <linux/completion.h> 中。通过以下宏静态地创建完成变量并初始化它：

```
DECLARE_COMPLETION(mr_comp);
```

通过 init_completion() 动态创建并初始化完成变量。

在一个指定的完成变量上，需要等待的任务调用 wait_for_completion() 来等待特定事件。当特定事件发生后，产生事件的任务调用 complete() 来发送信号唤醒正在等待的任务。表 10-9 列出了完成变量的方法。

表 10-9 完成变量方法

方 法	描 述
<code>init_completion(struct completion *)</code>	初始化指定的动态创建的完成变量
<code>wait_for_completion(struct completion *)</code>	等待指定的完成变量接收信号
<code>complete(struct completion *)</code>	发信号唤醒任何等待任务

使用完成变量的例子可以参考 `kernel/sched.c` 和 `kernel/fork.c`。完成变量的通常用法是，将完成变量作为数据结构中的一项动态创建，而完成数据结构初始化工作的内核代码将调用 `wait_for_completion()` 进行等待。初始化完成后，初始化函数调用 `completion()` 唤醒在等待的内核任务。

10.8 BLK：大内核锁

欢迎来到内核的原始混沌时期。BKL（大内核锁）是一个全局自旋锁，使用它主要是为了方便实现从 Linux 最初的 SMP 过渡到细粒度加锁机制。我们下面来介绍 BKL 的一些有趣的特性：

- 持有 BKL 的任务仍然可以睡眠。因为当任务无法被调度时，所加锁会自动被丢弃；当任务被调度时，锁又会被重新获得。当然，这并不是说，当任务持有 BKL 时，睡眠是安全的，仅仅是可以这样做，因为睡眠不会造成任务死锁。
- BKL 是一种递归锁。一个进程可以多次请求一个锁，并不会像自旋锁那样产生死锁现象。
- BKL 只可以用在进程上下文中。和自旋锁不同，你不能在中断上下文中申请 BLK。
- 新的用户不允许使用 BLK。随着内核版本的不断前进，越来越少的驱动和子系统再依赖于 BLK 了。

这些特性有助于 2.0 版本的内核向 2.2 版本过渡。在 SMP 支持被引入到 2.0 版本时，内核中一个时刻上只能有一个任务运行（当然，经过长期发展，现在内核已经被很好地线程化了）。2.2 版本的目标是允许多处理器在内核中并发执行程序。引入 BKL 是为了使到细粒度加锁机制的过渡更容易些，虽然当时 BKL 对内核过渡很有帮助，但是目前它已成为内核可扩展性的障碍了。

在内核中不鼓励使用 BKL。事实上，新代码中不再使用 BKL，但是这种锁仍然在部分内核代码中得到沿用，所以我们仍然需要理解 BKL 以及它的接口。除了前面提到的以外，BKL 的使用方式和自旋锁类似。函数 `lock_kernel()` 请求锁，`unlock_kernel()` 释放锁。一个执行线程可以递归的请求锁，但是，释放锁时也必须调用同样次数的 `unlock_kernel()` 操作，在最后一个解锁操作完成后，锁才会被释放。函数 `kernel_locked()` 检测锁当前是否被持有，如果被持有，返回一个非 0 值，否则返回 0。这些接口被声明在文件 `<linux/smp_lock.h>` 中，简单的用法如下：

```
lock_kernel();
/* 临界区，对所有其他的 BLK 用户进行同步……
 * 注意，你可以安全地在此睡眠，锁会悄无声息地被释放
 * 当你的任务被重新调度时，锁又会被悄无声息地获取
 * 这意味着你不会处于死锁状态，但是，如果你需要锁保护这里的数据
 * 你还是不需要睡眠
 */
unlock_kernel();
```

BKL 在被持有时同样会禁止内核抢占。在单一处理器内核中，BKL 并不执行实际的加锁操作。表 10-10 列出了所有 BKL 函数。

表 10-10 BKL 函数列表

函 数	描 述
lock_kernel()	获得 BKL
unlock_kernel()	释放 BKL
kernel_locked()	如果锁被持有返回非 0 值，否则返回 0（UP 总是返回非 0）

对于 BKL 最主要的问题是确定 BKL 锁保护的到底是什么。多数情况下，BKL 更像是保护代码（如“它保护对 foo() 函数的调用者进行同步”）而不保护数据（如“保护结构 foo”）。这个问题给利用自旋锁取代 BKL 造成了很大困难，因为难以判断 BKL 到底锁的是什麼，更难的是，发现所有使用 BKL 的用户之间的关系。

10.9 顺序锁

顺序锁，通常简称 seq 锁，是在 2.6 版本内核中才引入的一种新型锁。这种锁提供了一种很简单的机制，用于读写共享数据。实现这种锁主要依靠一个序列计数器。当有疑义的数据被写入时，会得到一个锁，并且序列值会增加。在读取数据之前和之后，序列号都被读取。如果读取的序列号值相同，说明在读操作进行的过程中没有被写操作打断过。此外，如果读取的值是偶数，那么就表明写操作没有发生（要明白因为锁的初值是 0，所以写锁会使值成奇数，释放的时候变成偶数）。

定义一个 seq 锁：

```
seqlock_t mr_seq_lock = DEFINE_SEQLOCK(mr_seq_lock);
```

然后，写锁的方法如下：

```
write_seqlock(&mr_seq_lock);
/* 写锁被获取 ... */
write_sequnlock(&mr_seq_lock);
```

这和普通自旋锁类似。不同的情况发生在读时，并且与自旋锁有很大不同：

```
unsigned long seq;
do{
    seq = read_seqbegin(&mr_seq_lock);
    /* 读这里的数据 ... */
}while(read_seqretry(&mr_seq_lock, seq));
```

在多个读者和少数写者共享一把锁的时候，seq 锁有助于提供一种非常轻量级和具有可扩展性的外观。但是 seq 锁对写者更有利。只要没有其他写者，写锁总是能够被成功获得。读者不会影响写锁，这点和读-写自旋锁及信号量一样。另外，挂起的写者会不断地使得读操作循环（前一个例子），直到不再有任何写者持有锁为止。

Seq 锁在你遇到如下需求时将是最理想的选择：

- 你的数据存在很多读者。
- 你的数据写者很少。
- 虽然写者很少，但是我希望写优先于读，而且不允许读者让写者饥饿。
- 你的数据很简单，如简单结构，甚至是简单的整型——在某些场合，你是不能使用原子量的。

使用 seq 锁中最有说服力的是 jiffies。该变量存储了 Linux 机器启动到当前的时间（参见第 11 章。Jiffies 是使用一个 64 位的变量，记录了自系统启动以来的时钟节拍累加数。对于那些能自动读取全部 64 位 jiffies_64 变量的机器来说，需要用 get_jiffies_64() 方法完成，而该方法的实现就是用了 seq 锁：

```
u64 get_jiffies_64(void)
{
    unsigned long seq;
    u64 ret;

    do {
        seq = read_seqbegin(&xtime_lock);
        ret = jiffies_64;
    } while (read_seqretry(&xtime_lock, seq));
    return ret;
}
```

定时器中断会更新 jiffies 的值，此刻，也需要使用 seq 锁变量：

```
write_seqlock(&xtime_lock);
jiffies_64 += 1;
write_sequnlock(&xtime_lock);
```

若要进一步了解 jiffies 和内核时间管理，请看第 11 章和内核源码树中的 kernel/timer.c 与 kernel/time/tick-common.c 文件。

10.10 禁止抢占

由于内核是抢占性的，内核中的进程在任何时刻都可能停下来以便另一个具有更高优先权的进程运行。这意味着一个任务与被抢占的任务可能会在同一个临界区内运行。为了避免这种情况，内核抢占代码使用自旋锁作为非抢占区域的标记。如果一个自旋锁被持有，内核便不能进行抢占。因为内核抢占和 SMP 面对相同的并发问题，并且内核已经是 SMP 安全的（SMP-safe），所以，这种简单的变化使得内核也是抢占安全的（preempt-safe）。

或许这就是我们希望的。实际中，某些情况并不需要自旋锁，但是仍然需要关闭内核抢占。最频繁出现的情况就是每个处理器上的数据。如果数据对每个处理器是唯一的，那么，这样的数据可能就不需要使用锁来保护，因为数据只能被一个处理器访问。如果自旋锁没有被持有，内核又是抢占式的，那么一个新调度的任务就可能访问同一个变量，如下所示：

```
任务 A 对每个处理器中未被锁保护的变量 foo 进行操作
任务 A 被抢占
```



```

任务 B 被调度
任务 B 操作变量 foo
任务 B 完成
任务 A 被调度
任务 A 继续操作变量 foo

```

这样，即使这是一个单处理器计算机，变量 `foo` 也会被多个进程以伪并发的方式访问。通常，这个变量会请求得到一个自旋锁（防止多处理器机器上的真并发）。但是如果这是每个处理器上独立的变量，可能就不需要锁。

为了解决这个问题，可以通过 `preempt_disable()` 禁止内核抢占。这是一个可以嵌套调用的函数，可以调用任意次。每次调用都必须有一个相应的 `preempt_enable()` 调用。当最后一次 `preempt_enable()` 被调用后，内核抢占才重新启用。例如：

```

preempt_disable();
/* 抢占被禁止 ... */
preempt_enable();

```

抢占计数存放着被持有锁的数量和 `preempt_disable()` 的调用次数，如果计数是 0，那么内核可以进行抢占；如果为 1 或更大的值，那么，内核就不会进行抢占。这个计数非常有用——它是一种对原子操作和睡眠很有效的调试方法。函数 `preempt_count()` 返回这个值。表 10-11 列出了内核抢占相关的函数。

表 10-11 内核抢占的相关函数

函 数	描 述
<code>preempt_disable()</code>	增加抢占计数值，从而禁止内核抢占
<code>preempt_enable()</code>	减少抢占计数，并当该值降为 0 时检查和执行被挂起的需调度的任务
<code>preempt_enable_no_resched()</code>	激活内核抢占但不再检查任何被挂起的需调度任务
<code>preempt_count()</code>	返回抢占计数

为了用更简洁的方法解决每个处理器上的数据访问问题，可以通过 `get_cpu()` 获得处理器编号（假定是用这种编号来对每个处理器的数据进行索引的）。这个函数在返回当前处理器号前首先会关闭内核抢占。

```

int cpu ;
/* 禁止内核抢占，并将 CPU 设置为当前处理器 */
cpu= get_cpu();
/* 对每个处理器的数据进行操作 ... */
/* 再给予内核抢占性，“CPU”可改变故它不再有效 */
put_cpu();

```

10.11 顺序和屏障

当处理多处理器之间或硬件设备之间的同步问题时，有时需要在你的程序代码中以指定的顺序发出读内存（读入）和写内存（存储）指令。在和硬件交互时，时常需要确保一个给定的读操

作发生在其他读或写操作之前。另外，在多处理器上，可能需要按写数据的顺序读数据（通常确保后来以同样的顺序进行读取）。但是编译器和处理器为了提高效率，可能对读和写重新排序^①，这样无疑使问题复杂化了。幸好，所有可能重新排序和写的处理器提供了机器指令来确保顺序要求。同样也可以指示编译器不要对给定点周围的指令序列进行重新排序。这些确保顺序的指令称作屏障（barriers）。

基本上，在某些处理器上存在以下代码：

```
a=1;
b=2;
```

有可能会在 a 中存放新值之前就在 b 中存放新值。

编译器和处理器都看不出 a 和 b 之间的关系。编译器会在编译时按这种顺序编译，这种顺序会是静态的，编译的目标代码就只把 a 放在 b 之前。但是，处理器会重新动态排序，因为处理器在执行指令期间，会在取指令和分派时，把表面上看似无关的指令按自认为最好的顺序排列。大多数情况下，这样的排序是最佳的，因为 a 和 b 之间没有明显的关系。尽管有些时候程序员知道什么是最好的顺序。

尽管前面的例子可能被重新排序，但是处理器和编译器绝不会对下面的代码重新排序：

```
a = 1;
b = a;
```

此处 a 和 b 均为全局变量，因为 a 与 b 之间有明确的数据依赖关系。

但是不管是编译器还是处理器都不知道其他上下文中的相关代码。偶然情况下，有必要让写操作被其他代码识别，也让所期望的指定顺序之外的代码识别。这种情况常常发生在硬件设备上，但是在多处理器机器上也很常见。

rmb() 方法提供了一个“读”内存屏障，它确保跨越 rmb() 的载入动作不会发生重排序。也就是说，在 rmb() 之前的载入操作不会被重新排在该调用之后，同理，在 rmb() 之后的载入操作不会被重新排在该调用之前。

wmb() 方法提供了一个“写”内存屏障，这个函数的功能和 rmb() 类似，区别仅仅是它是针对存储而非载入——它确保跨越屏障的存储不发生重排序。

mb() 方法既提供了读屏障也提供了写屏障。载入和存储动作都不会跨越屏障重新排序。这是因为一条单独的指令（通常和 rmb() 使用同一个指令）既可以提供载入屏障，也可以提供存储屏障。

read_barrier_depends() 是 rmb() 的变种，它提供了一个读屏障，但是仅仅是针对后续读操作所依靠的那些载入。因为屏障后的读操作依赖于屏障前的读操作，因此，该屏障确保屏障前的读操作在屏障后的读操作之前完成。明白了吗？基本上说，该函数设置一个读屏障，如 rmb()，但是只针对特定的读——也就是那些相互依赖的读操作。在有些体系结构上，read_barrier_depends() 比 rmb() 执行得快，因为它仅仅是个空操作，实际并不需要。

① 虽然 Intel x86 处理器不会对写进行重新排序，也就是说，它不进行打乱顺序的存储，但是其他处理器会这么做。